

Quaderno di lavoro

Che cosa è cambiato e perché

Settembre 2026 — dal laboratorio emulato
al datapath reale

Progetto **IPA/eBPF**

inferenza di una rete neurale dentro il kernel, a bordo del pacchetto

Ogni numero di questo quaderno viene da una misura eseguita,
non da una stima. Dove una cosa non è stata verificata, è scritto.

Indice

1	Di che cosa stiamo parlando	4
1.1	L'idea in una pagina	4
1.2	Dove gira: XDP	4
1.3	Le tre pipeline	4
2	La matematica che sta dentro il kernel	5
2.1	Perché i numeri sono interi	5
2.2	Il problema dei bias, e l'errore che ci stava dentro	5
2.3	Il TTL: dividere il prodotto, mai il valore	6
3	Le classi non sono porte	7
3.1	Il problema	7
3.2	La soluzione: una catena esplicita	7
3.3	Un errore che questa separazione ha fatto emergere	8
3.4	Ogni modello ha la sua semantica (24 settembre)	8
4	Il motore non è lo scenario	10
4.1	Com'era	10
4.2	Com'è adesso: tre strati	10
4.3	La prova	11
5	Il modello non è quello depositato	12
5.1	La domanda	12
5.2	I modelli sintetici	12
5.3	Che cosa era già provato, e che cosa no	12
5.4	Il confronto a quattro vie	13
5.5	Il risultato	13
5.6	Ma ci è arrivato trovando due difetti	14
5.7	La correzione	14
5.8	Che cosa è costata, e la prova che regge	15
5.9	Tre difetti, e chi li ha trovati	15
5.10	Che cosa questo capitolo dimostra	16
6	Dall'emulatore al kernel vero	17
6.1	Che cosa provavano i test di prima	17
6.2	Il banco nuovo	17
6.3	Tre difetti del test, trovati misurando	17
6.4	Il risultato	18
7	La feature che non c'era	19
7.1	Il sintomo	19
7.2	Perché nessun test lo vedeva	19
7.3	La correzione	19
7.4	La prova che adesso è viva	20
7.5	Il prezzo, misurato	20

8	Il nodo non sapeva di essere un nodo	21
8.1	L'ingresso più grande di tutti	21
8.2	Il sintomo	21
8.3	La correzione, e perché una tabella e non un array	21
8.4	Una pipeline ha resistito un giorno	21
8.5	La prova che il nodo è vivo	22
9	Il verificatore non conta le istruzioni	23
9.1	Due monete diverse	23
9.2	Il ciclo era girato dalla parte sbagliata	23
9.3	Perché il risultato non cambia di un bit	23
9.4	Il risultato, che sembra un errore	23
9.5	Un effetto collaterale che vale una riga di tabella	24
10	I numeri, e che cosa dicono davvero	25
10.1	La tabella	25
10.2	Primo risultato: la dimensione non predice la velocità	25
10.3	Secondo risultato: quanto valgono le latenze assolute	26
10.4	Terzo risultato: la genericità ha un costo di verifica	27
10.5	Quarto risultato: togliere il compilatore dal nodo	27
10.6	Che cosa questi numeri non dicono	28
11	Come cresce il costo	29
11.1	La domanda	29
11.2	Come è impostata la misura	29
11.3	Profondità: chi paga in spazio e chi in tempo	30
11.4	Aggiornare il modello: con l'oggetto precompilato, pari	31
11.5	Larga o profonda? Dipende da chi la esegue	32
11.6	Congelare il nodo: conviene?	34
11.7	Congelare quali porte esistono	38
11.8	Due larghezze, due risposte opposte	40
11.9	I pesi contano, ma solo per una delle tre	42
11.10	Il risultato che non cercavamo	42
11.11	Che cosa questi grafici non dicono	43
12	Il costo del programma, misurato da solo	44
12.1	La campagna sulle architetture	44
12.2	Il problema: le MAC nominali non predicono la latenza	45
12.3	La soluzione: contare le MAC che vengono eseguite	45
12.4	Un difetto del banco, trovato dalla calibrazione	46
13	Il test reale: quanto costa un pacchetto al nodo	49
13.1	La domanda, in parole semplici	49
13.2	Com'è fatto il banco	49
13.3	Le cinque trappole	50
13.4	Il risultato: quanto costa un pacchetto	53
13.5	Quanto è solido questo numero	54
13.6	Quello che su questa macchina non si può risolvere	56
13.7	Che cosa questo numero non dice	56
13.8	Le cifre vecchie e quelle nuove	57

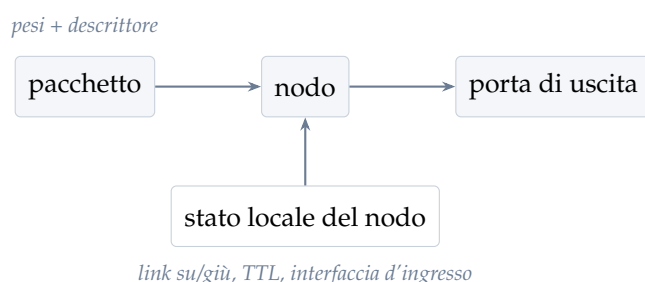
14 Le altre correzioni di settembre	58
14.1 Il P1 che va sui nodi non inoltrava	58
14.2 Tutti i test di P1 usano ora l'oggetto AOT	58
14.3 Il cronometro del caricatore AOT	58
14.4 Due implementazioni che devono dare lo stesso numero	59
14.5 La scala delle feature in P2 e P3	59
14.6 Pulizia del repository	60
15 I numeri di oggi, in una pagina	61
16 Quel che resta	63
16.1 Fatto	63
16.2 Aperto	64
16.3 Il filo conduttore	64
17 I comandi	66
17.1 Correttezza: il modello fa quello che deve	66
17.2 Kernel: si carica, e quanto costa	66
17.3 Datapath reale: il pacchetto attraversa davvero	67
17.4 Analisi parametrica: come cresce il costo	67
17.5 Il test reale: il costo del nodo	68
17.6 Costi specifici	69
17.7 Oggetto precompilato (AOT)	69
17.8 Diagnostica, quando qualcosa non torna	69
17.9 L'ordine consigliato	69

Di che cosa stiamo parlando

1.1 L'idea in una pagina

Un router normale decide dove mandare un pacchetto consultando una tabella riempita da un protocollo di routing. L'idea di **IPA** (*Intelligent PAckets*) è diversa: il pacchetto **porta con sé** una piccola rete neurale, e ogni nodo che attraversa la esegue per decidere l'uscita.

Niente segnalazione, niente tabella condivisa: la decisione nasce dai dati che il nodo ha sotto mano in quel momento.



Il modello è una rete piccola: quattro tipi di ingresso, due strati nascosti da quattro neuroni, sette uscite. In tutto **319 pesi**. Deve stare in un header ed essere eseguibile in decine di nanosecondi.

1.2 Dove gira: XDP

Il codice gira in **eBPF** agganciato a **XDP**, il punto più precoce in cui il kernel Linux permette di toccare un pacchetto: prima che venga costruita la struttura `sk_buff`, quindi prima di quasi tutto il resto dello stack di rete.

Questo impone un vincolo forte, che spiega metà delle scelte di progetto: un programma eBPF deve essere **dimostrato sicuro** da un componente del kernel chiamato *verificatore* prima di poter girare. Niente cicli di lunghezza ignota, niente accessi non controllati alla memoria, niente ricorsione. Tutto ciò che nel documento sembra contorto — cicli srotolati, tetti dichiarati a priori, tabelle al posto di calcoli — nasce da lì.

1.3 Le tre pipeline

Lo stesso modello è implementato in tre modi, per misurare un compromesso.

	i pesi stanno	cambiare modello
P1 hardcoded	scritti nel codice C	ricompilare
P2 template	in una mappa	scrivere la mappa
P3 modular	in una mappa, uno strato per programma	scrivere la mappa

P1 è veloce perché il compilatore vede i pesi e li semplifica; è rigido perché ogni modello nuovo richiede una ricompilazione. P2 e P3 sono l'opposto. Il lavoro di settembre serve a rendere questo confronto *misurabile davvero*.

La matematica che sta dentro il kernel

2.1 Perché i numeri sono interi

Nel kernel non ci sono numeri in virgola mobile: eBPF lavora con interi. Il modello però è addestrato in virgola mobile. Serve una conversione, e si chiama **quantizzazione**.

Ogni peso reale w diventa un intero a 8 bit:

$$w_{\text{int}} = \text{round}(w \cdot s)$$

dove s è il **fattore di scala**, uno solo per tutta la rete.

Perché il risultato stia in un byte con segno serve

$$s \cdot \max_i |w_i| \leq 127.$$

Sul modello depositato $\max |w| = 5,098$, quindi $s \leq 24,9$. Due scelte convivono nel progetto: $s = 24$ (il massimo intero ammesso) e $s = 16$ (la potenza di due più vicina per difetto, che rende le divisioni degli shift).

Superare quel limite non dà un errore: dà un *overflow silenzioso*. I pesi grandi si troncano e il modello sbaglia senza che nulla lo segnali. Per questo i test distinguono le scale *rappresentabili* da quelle che saturano, e misurano solo sulle prime.

2.2 Il problema dei bias, e l'errore che ci stava dentro

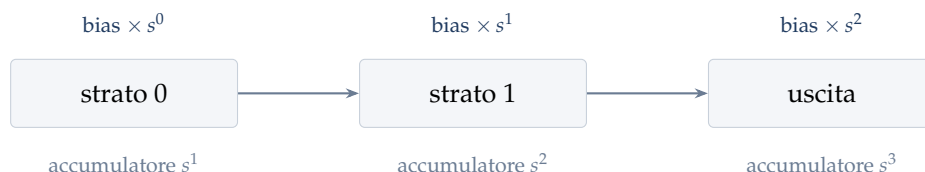
Qui sta l'errore più sottile trovato in settembre, e vale la pena seguirlo passo passo perché è istruttivo.

Quando moltiplichiamo un ingresso per un peso quantizzato, il risultato porta un fattore s in più rispetto al valore reale:

$$x \cdot w_{\text{int}} = x \cdot w \cdot s.$$

Se impiliamo gli strati, quel fattore si accumula. Dopo $\ell + 1$ moltiplicazioni l'accumulatore dello strato ℓ porta $s^{\ell+1}$.

Ma il **bias** è memorizzato come un peso qualsiasi, cioè con un solo fattore s . Sommarlo così com'è significa sommare una quantità sistematicamente troppo piccola. Va riportato nelle unità dell'accumulatore:



La regola è semplice: allo strato ℓ (contando da zero) il bias va moltiplicato per s^ℓ .

L'errore

Il modello di riferimento in Python — quello con cui si controlla che il datapath calcoli la cosa giusta — applicava s^0 e s^1 correttamente ai primi due strati, e lasciava l'**ultimo strato** a s^0 invece di s^2 .

Il bias dell'uscita era sottopesato di $s^2 = 24^2 = 576$.

Perché nessuno se n'era accorto per mesi

Tutti i test eseguivano il modello con **tutti i link attivi**. In quella condizione gli accumulatori sono grandi e un errore di 576 non sposta mai il massimo. Basta spegnere qualche link perché gli accumulatori si riducano e l'errore decida l'esito.

Nello stesso file c'era una seconda funzione di riferimento, scritta per forme arbitrarie, che usava la formula generale s^ℓ ed era **già corretta**. Due riferimenti, uno giusto e uno sbagliato, e quello sbagliato era l'unico mai esercitato con link spenti.

Corretto l'errore, il riferimento e le tre pipeline concordano su tutti i casi provati.

La lezione non è "c'era un bug". È che **un test che varia un solo ingresso non è un test**: verifica una condizione e la ripete. Ci torneremo nel capitolo 5, dove lo stesso schema si ripresenta in una forma ancora più netta.

2.3 Il TTL: dividere il prodotto, mai il valore

Il modello è addestrato con il TTL *normalizzato*, cioè ttl/T con $T = 30$ il valore iniziale. Con i numeri interi

$$\frac{\text{ttl}}{30} = 0 \quad \text{per ogni } \text{ttl} < 30,$$

e la feature sparirebbe. La divisione va quindi applicata al *prodotto*:

$$\left\lfloor \frac{\text{ttl} \cdot w_{\text{int}}}{T} \right\rfloor \quad \text{e non} \quad \left\lfloor \frac{\text{ttl}}{T} \right\rfloor \cdot w_{\text{int}}.$$

Nel C generato si legge esattamente così:

```
(((__s64)_ttl * 11LL) / 30LL)
```

Le classi non sono porte

3.1 Il problema

La rete produce sette numeri; il più grande vince. Ma *che cosa significa* la classe vincente?

All'inizio di settembre la risposta era sparsa e incoerente. In punti diversi del progetto convivevano quattro convenzioni su quale classe significasse "scarta il pacchetto":

Convenzione	Dove
la classe 6 è DROP	in un punto
la classe 5 è DROP	in un altro
$\text{DROP} = n_{\text{out}} - 1$	in un terzo
$\text{DROP} = \max(\text{etichette}) + 1$	dedotto dai dati

E sotto ce n'era una quinta, implicita e più insidiosa: che la classe k volesse dire "manda fuori dall'interfaccia k ". Funzionava per caso, perché nel modello depositato le porte sono numerate come le classi.

3.2 La soluzione: una catena esplicita

Adesso il significato è **dichiarato**, mai dedotto, e passa per una catena di quattro passi, ognuno con un responsabile diverso:



- **classe**: quello che il modello ha calcolato;
- **azione**: FORWARD, DROP o UNUSED — una classe mai addestrata esiste ed è dichiarata tale;
- **porta logica**: un numero che il *modello* conosce;
- **ifindex**: l'interfaccia vera, che solo il *nodo* conosce e che è diversa su ogni macchina.

Nel modello depositato la dichiarazione è:

classe	azione	porta logica
0-4	FORWARD	0-4
5	DROP	—
6	UNUSED	—

Il punto non è che adesso funziona: funzionava anche prima, per coincidenza numerica. Il punto è che adesso **smetterebbe di funzionare rumorosamente** se il modello cambiasse, invece di continuare a girare dando risposte plausibili e sbagliate.

3.3 Un errore che questa separazione ha fatto emergere

Il datapath teneva un contatore per classe, ma scriveva nella casella della *porta* invece che in quella della *classe*. Sembrava giusto perché le due numerazioni coincidono nel modello depositato.

Con un modello in cui non coincidono, il contatore avrebbe misurato la cosa sbagliata in silenzio. Separare i due spazi di indici ha reso l'errore visibile e correggibile.

3.4 Ogni modello ha la sua semantica (24 settembre)

Il limite. In P2 e P3 la tabella classe $\rightarrow$ azione era *una*, letta con la sola classe e condivisa da tutti i modelli caricati. Due modelli non potevano dare alla stessa classe significati diversi: per evitare che il secondo riscrisse in silenzio le azioni del primo, il piano di controllo lo rifiutava. Il vantaggio di P2 e P3 — un programma solo, i modelli caricati come dati — valeva quindi solo per modelli con la stessa semantica.

La soluzione. La chiave della tabella è diventata la coppia (model_id, classe). Il programma resta uno per architettura; aggiungere un modello resta scrivere delle tabelle.



- **Una tabella ad accesso diretto, non una tabella hash:** 256 modelli $\times$ 2 banchi $\times$ 32 classi. Resta una sola lettura per pacchetto, come prima, e nessun ciclo sui modelli registrati.
- **Due banchi per modello.** Ricaricare un modello scrive il banco inattivo, poi un'unica scrittura del registro cambia insieme il numero di classi e il banco in uso. Un pacchetto vede la semantica vecchia o quella nuova, mai una tabella scritta a metà.
- **Perché non dentro il registro,** che il programma ha già letto: usare la classe come indice dentro quel valore costringe il verificatore a seguirne il valore esatto attraverso tutto l'argmax, e smette di potare i cammini. Misurato: layer_hidden di P3 non si carica più.
- **Senza un valore vivo in più.** Il primo strato di P3 sta al limite dei 512 byte di stack nelle sue versioni strumentate (quella che conta le letture e quella con i timestamp T1/T2 del test reale): tenere vivi fino alla fine model_id e il registro la faceva sforare. La chiave viaggia allora nella variabile che già diceva «questo è l'ultimo strato», e il programma cresce di 9 istruzioni.

La prova. Modelli costruiti in modo che il TTL scelga la classe, così ogni classe di ogni modello è raggiungibile: un modello solo, due con la stessa semantica, due con semantiche opposte su ogni classe a pacchetti alternati, un modello inesistente o rimosso, numeri di classi diversi, un modello ricaricato con un'altra semantica. **53/53** su P2 e P3. Con una tabella condivisa simulata gli stessi test falliscono dove devono: il test distingue le due cose.

	P2		P3	
	classe sola	per modello	classe sola	per modello
istruzioni	14 985	14 976	12 349	12 374
letture di tabella	11	11	29	29
latenza, mediana	289 ns	298 ns	477 ns	489 ns
memoria delle tabelle	11 192 B	142 264 B	20 284 B	151 356 B

Il datapath non se ne accorge: stesse letture, pochi byte di codice in più o in meno, latenza entro il rumore, e nessuna crescita da 1 a 8 modelli. Il prezzo è in memoria: 128 KiB fissi per pipeline, riservati per 256 modelli anche quando ne è caricato uno solo.

Il motore non è lo scenario

4.1 Com'era

Il progetto era nato attorno a una topologia precisa: **Germany50**, una rete di 50 router usata come riferimento in letteratura, con due host attaccati, emulata con **Kathara** (container Docker connessi a formare la rete).

Due numeri erano ovunque nel codice:

$$n_{\text{nodi}} = 52, \quad n_{\text{interfacce}} = 6.$$

Il primo è $50 + 2$. Il secondo è il **grado massimo** della rete: il nodo con più vicini. Non è il grado di un nodo qualsiasi — tutti gli altri riempiono di zeri gli slot che non usano.

Questi numeri determinano la larghezza dell'ingresso:

$$n_{\text{in}} = \underbrace{6}_{\text{link}} + \underbrace{6}_{\text{iface}} + \underbrace{1}_{\text{ttl}} + \underbrace{52}_{\text{nodo}} = 65.$$

Il problema non era che esistessero, ma che il motore li conoscesse. C'era un *default* interno: se nessuno diceva niente, il motore assumeva 6 e 52. Ogni chiamante che dimenticava di specificare lo scenario risultava silenziosamente corretto per Germany50 e silenziosamente sbagliato altrove.

4.2 Com'è adesso: tre strati



La dipendenza punta **solo verso l'alto**. Il motore non sa che cosa sia Germany50.

Le dimensioni si risolvono in ordine: una variabile d'ambiente, poi un file di sistema, poi il blocco `trained_on` del descrittore del modello. Se nessuna delle tre risponde, il motore **si ferma** e dice dove ha cercato.

Un default comodo è il modo più efficace di nascondere un'assunzione. Preferire un errore esplicito a un valore plausibile è una scelta di progetto, non pignoleria: il valore plausibile non si manifesta mai come errore, solo come risultato sbagliato mesi dopo.

4.3 La prova

Non basta dirlo. Il motore genera oggi un datapath corretto per cinque topologie che non hanno niente a che vedere con quella d'origine:

iface	nodi	strati	n_{in}	n_{out}	vettore link
3	8	4-4	15	4	v[3]
4	16	6	25	5	v[4]
5	24	4-4-4	35	6	v[5]
2	6	3-3	11	3	v[2]
6	52	4-4	65	7	v[6]

Nessuna riga di C scritta a mano cambia fra una riga e l'altra.

Il modello non è quello depositato

5.1 La domanda

Il capitolo precedente ha tolto la topologia dalle assunzioni del codice. Restava dentro un'assunzione più silenziosa: **un solo modello**. Ogni misura, ogni confronto fra pipeline e ogni prova di correttezza girava sullo stesso checkpoint fornito dal relatore — 65-4-4-7, scala 25.

Il rischio non è teorico. Un datapath che calcola la cosa giusta su un modello può calcolarla per caso: perché quel modello ha una forma particolare, perché i suoi pesi cadono in un intervallo comodo, o perché una costante scritta nel codice coincide con un suo parametro. Finché il modello è uno solo, la differenza fra “corretto” e “corretto qui” non si vede.

5.2 I modelli sintetici

La risposta sono **sette scenari generati**, con pesi casuali e nessun rapporto con il checkpoint depositato. Coprono un asse ciascuno:

Scenario	Forma	Pesi
ones	11-2-3	33
small	15-4-4	84
mixed	18-6-5	149
ipa_like	65-4-4-7	319
sparse	65-4-4-7	319
deep	65-4-4-4-7	339
large	117-8-8-9	1 097

Il generatore non produce solo numeri a caso: rispetta la **struttura** delle feature, che è la parte che un datapath può sbagliare. Ogni feature dichiara il suo tipo — `real` (un valore in un intervallo, che il datapath porta come intero diviso per una scala), `integer`, `binary` (un vettore di 0/1), `onehot` (esattamente un 1) — con i suoi estremi, la sua scala e i suoi vincoli. Ogni scenario porta con sé **1 000 vettori d'ingresso**, nella versione intera e in quella a virgola mobile, e le predizioni attese per entrambe.

Gli artefatti non stanno in git: sono rigenerabili byte per byte a parità di seme, e il test li rigenera da solo se non li trova.

5.3 Che cosa era già provato, e che cosa no

Su quei sette scenari il progetto verificava che i pesi **compilassero** e passassero il verificatore del kernel, e che i control plane accettassero i blocchi. È una prova reale ma di una cosa sola: che il programma esiste. Non dice *quale classe sceglie*.

La distinzione che mancava

Un programma che compila e gira può comunque decidere diversamente dal modello che dovrebbe eseguire. La domanda scoperta era: *a parità di ingresso, il datapath sceglie la stessa classe del riferimento?*

5.4 Il confronto a quattro vie

Due confronti non bastano, perché un disaccordo ha più di una causa possibile e le cause chiedono rimedi diversi. Le vie sono quattro:

Confronto	Che cosa isola
float vs int8 (Python)	l'errore di quantizzazione : il modello a interi decide diversamente da quello a virgola mobile. Può legittimamente non essere 100%.
int8 (Python) vs int8 (riferimento)	controllo : due implementazioni intere indipendenti del progetto devono concordare fra loro, o non è chiaro quale sia il riferimento.
int8 (riferimento) vs int8 (eBPF)	se il datapath è esatto rispetto alla configurazione che gli è stata data .
int8 (Python) vs int8 (eBPF)	l'errore di implementazione . Deve valere 100%: un solo disaccordo qui è un difetto del datapath.

Perché non i 1 000 vettori depositati

Il datapath non accetta un vettore d'ingresso arbitrario: lo *costruisce* da ciò che ha sotto mano — le mappe dense, il TTL del pacchetto, la porta d'ingresso risolta, l'indice del nodo. Un one-hot node diverso a ogni campione vorrebbe dire ricompilare il programma a ogni campione.

Si campiona quindi nello spazio che il datapath sa davvero esprimere, e il vettore Python si costruisce allo stesso modo. Confrontare due implementazioni su ingressi che una delle due non potrebbe mai ricevere non direbbe niente su nessuna delle due.

5.5 Il risultato

Trecento ingressi per scenario, seme fisso, pipeline P1.

Scenario	float vs int8	int8 Python vs eBPF	decisioni
ones	100,00 %	100,00 %	300/300
small	98,00 %	100,00 %	300/300
mixed	100,00 %	100,00 %	300/300
ipa_like	97,67 %	100,00 %	300/300
sparse	99,33 %	100,00 %	300/300
deep	98,67 %	100,00 %	300/300
large	99,67 %	100,00 %	300/300

Sette scenari su sette, equivalenza numerica esatta. La colonna di sinistra — fra 97,67 % e 100 % — è la quantizzazione, cioè una proprietà del modello e della scala scelta, non un difetto.

5.6 Ma ci è arrivato trovando due difetti

Il valore del test non sta nella tabella: sta nel fatto che la prima esecuzione **non** dava 7/7. Dava 1/7, e le due cause erano di natura diversa.

Il primo: il banco

Cinque scenari su sette stavano fra il 71 % e l'85 %. Il banco non comunicava al kernel la porta d'ingresso: `prog_test_run` accetta un parametro `ingress_ifindex` ma lo **ignora** — gira senza `ctx_in`, e il programma vede sempre l'`ifindex` del dispositivo di prova del kernel. Il riferimento Python usava una porta variabile, il datapath ne vedeva sempre nessuna, e le due metà costruivano due ingressi diversi.

Il segnale diagnostico è stato più utile del sintomo: dopo la prima correzione le percentuali non si erano mosse **di un centesimo**. Codice che non cambia niente non è codice sbagliato: è codice mai eseguito. Confermato senza kernel, simulando “il datapath vede porta 0”: le percentuali osservate si riproducono con scarto **0,00** su sei scenari.

Il secondo: la scala compilata

Resisteva `small`, al 70,33 %. Con una coincidenza che era la risposta: le righe *int8 Python vs riferimento* e *int8 Python vs eBPF* davano lo stesso identico numero, e *riferimento vs eBPF* dava 100 %. Il datapath non sbagliava un conto: eseguiva **esattamente** una configurazione sbagliata.

Tutte e tre le pipeline fissavano la scala della feature `ttl` a **30** in fase di compilazione — `feature_scale()` in P1, `T2_TTL_SCALE` in P2, `ML_TTL_SCALE` in P3 — e la struttura del descrittore letto a runtime, `struct feat_ent`, non aveva un campo dove metterla. Un modello addestrato su `ttl/16` veniva eseguito con `ttl/30`. Gli scenari `small` (16), `large` (64) e `ones` (8) sono esattamente quei modelli.

Sul checkpoint depositato, che usa 30, il difetto è invisibile.

La genericità di P2 e P3 — “il descrittore si legge a runtime, quindi qualunque modello va bene” — era vera per la forma, il numero di feature e gli offset, e **falsa per la scala**: l'unico numero del descrittore che deve seguire il modello era l'unico compilato dentro. Un'affermazione di genericità va verificata voce per voce, non in blocco.

5.7 La correzione

La scala ora viaggia nel descrittore. Il campo c'era già: `struct feat_ent` aveva un byte di riempimento inutilizzato, e le scale in gioco (8, 16, 30, 64) ci stanno.

Dove	Che cosa cambia
<code>model_meta</code>	<code>feature_scale_of(feat)</code> risponde per <i>feature</i> e non per <i>tipo</i> ; la scala entra nel descrittore risolto.
P1	compila la scala del descrittore invece di quella del catalogo.
P2 / P3	<code>_pad</code> diventa <code>scale</code> ; la divisione la legge dalla mappa. Con 0 si ricade sul default compilato, quindi i descrittori scritti prima restano validi.

Il modello depositato risolve a 30 come prima: nessuna regressione.

Un effetto collaterale che il compilatore ha fermato

Passare da una costante a una variabile ha rotto la compilazione di P2: “*unsupported signed division*”. Finché il divisore era un `#define` clang lo trasformava in uno spostamento di bit; da variabile diventa una divisione vera, e **BPF non ha la divisione con segno**. Il numeratore qui ce l’ha, perché i pesi possono essere negativi.

La correzione divide il valore assoluto senza segno e rimette il segno, con troncamento verso lo zero — lo stesso arrotondamento del riferimento Python, perché sui negativi le due implementazioni coincidono solo se arrotondano allo stesso modo.

5.8 Che cosa è costata, e la prova che regge

Una correzione che tocca il codice che gira per ogni pacchetto va pagata e misurata, non solo dichiarata. La suite kernel, rieseguita sul modello depositato:

Pipeline	istruzioni prima	dopo	differenza	
baseline	155	155	invariata	
P1 hardcoded	1 026	1 026	invariata	
P2 template	14 628	14 985	+357	(+2,4 %)
P3 modular	12 031	12 349	+318	(+2,6 %)

Il costo è **due virgola cinque per cento** sulle due pipeline generiche: il codice della divisione a divisore variabile più la lettura della scala dalla mappa. Che baseline e P1 restino *identiche al conteggio* è la verifica che il cambiamento non ha toccato percorsi che non doveva toccare — P1 compila la scala come prima, solo presa da un posto diverso.

Perché la latenza non si può usare per giudicare

Nello stesso run la latenza si è spostata su tutte e quattro le colonne, **compresa la baseline**: da 29 a 35 ns, cioè +21 %, su un programma che non è stato modificato di una riga.

Quel +21 % è il metro del rumore della macchina in quell’esecuzione, e sta *sopra* lo spostamento di P2 (+9 %) e P3 (+5 %). Le differenze di latenza qui non sono attribuibili alla modifica: l’unica colonna che dice qualcosa è quella delle istruzioni, che è deterministica.

L’esito funzionale è kernel suite: PASS — le quattro pipeline, la gestione del TTL, il reroute su link caduto, le architetture alternative e i due modelli concorrenti registrati insieme su P2 e su P3. E nel registro del descrittore si legge la scala arrivare a destinazione:

```
model_desc[0] = feats=[
(1,6,0,1) (2,6,6,1) (3,1,12,30) (4,52,13,1)
```

Il quarto campo è la scala: `ttl` porta 30 fin dentro la mappa, e non è più un numero scritto nel sorgente.

5.9 Tre difetti, e chi li ha trovati

Vale la pena tenere il conto, perché dice qualcosa sul metodo più che sul codice.

Difetto	Di chi	Trovato da
Porta d'ingresso non detta al kernel	banco	il confronto, al primo run
Scala del TTL compilata a 30	datapath	il confronto, tolto il primo
Divisione con segno, non ammessa da BPF	correzione	il compilatore
Funzione nel ramo sbagliato di <code>#ifndef</code>	correzione	il compilatore, al secondo giro

Gli ultimi due sono stati introdotti *riparando* il secondo, e nessuno dei due sarebbe stato visibile senza compilare davvero. Da lì è nato `check_source_visibility.py`: i sorgenti eBPF di questo progetto non sono file ma stringhe, concatenate in combinazioni diverse dai chiamanti e ritagliate dal preprocessore con macro diverse. Una funzione può stare prima del suo uso *nel file* e sparire lo stesso nella combinazione che conta. Il controllo valuta le guardie e verifica la visibilità in tutte e cinque le combinazioni realmente compilate — senza root e senza BCC, quindi prima di spedire e non dopo.

5.10 Che cosa questo capitolo dimostra

Che le tre pipeline calcolano ciò che il modello dice, su sette modelli che nessuno ha scelto per farle funzionare. E, di passaggio, che l'affermazione era falsa prima di essere verificata — il che è il motivo per cui la verifica serviva.

Dall'emulatore al kernel vero

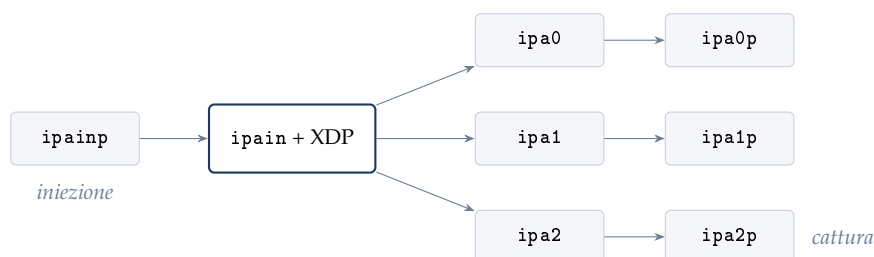
6.1 Che cosa provavano i test di prima

Fino a settembre ogni misura veniva da una chiamata di sistema, `BPF_PROG_TEST_RUN`: si dà al programma un buffer, lo si esegue N volte, e il kernel riporta quanti nanosecondi in media.

È utile e onesto per confrontare le tre pipeline fra loro. Ma non tocca un'interfaccia, non ha un driver, non consegna niente. Il programma poteva dire "redirigerei sulla porta 3" e nessuno verificava che un pacchetto ne uscisse davvero.

6.2 Il banco nuovo

Kathara è stato sostituito da qualcosa di più semplice e più vicino al vero: coppie di interfacce virtuali (veth) create sull'host.



Si inietta un frame da un capo, il programma XDP gira davvero, e si legge da quale interfaccia è uscito. Questo risponde alla domanda che `BPF_PROG_TEST_RUN` non può porre.

Due dettagli che sembrano minori e non lo sono

Modalità di attacco. XDP si può agganciare in due modi: *native*, dentro il driver, prima che esista la `sk_buff`; oppure *generic*, dopo. Il progetto usava sempre *generic*, e il motivo era scritto nel codice: serviva per le interfacce dentro i container dell'emulatore. Tolto l'emulatore, quel motivo cade. Adesso il default è *native*, la modalità *davvero in effetto* viene riletta dal kernel dopo l'aggancio, e un fallimento è un errore, non una retrocessione silenziosa.

Il vincolo delle veth. Redirigere *verso* una veth funziona solo se il lato ricevente ha a sua volta un programma XDP agganciato: il kernel alloca le code necessarie solo in quel caso. Senza, il redirect fallisce in silenzio e il pacchetto sparisce — indistinguibile da un modello che decide di scartare tutto. Il banco aggancia quindi un programma minimo a ogni capo.

6.3 Tre difetti del test, trovati misurando

Il banco nuovo ha trovato più difetti nei *test* che nel prodotto. Vale la pena elencarli, perché il motivo per cui erano invisibili è sempre lo stesso.

1. Il TTL non muove questo modello

La prima versione del test variava il TTL da 2 a 8 e riportava *"7 verifiche su 7 superate"*. Misurando lo spazio raggiungibile si scopre che con tutti i link attivi il modello risponde **sempre classe 2**, per qualsiasi TTL. Quel 7 su 7 era **un caso solo, ripetuto sette volte**.

È `link_state` la feature che sposta la decisione. Il test adesso *cerca* un ingresso per ogni classe, invece di spazzare alla cieca.

2. Un pacchetto del kernel contato come risultato

Su una veth appena creata il kernel manda del traffico suo (IPv6 multicast). Una volta è stato catturato e contato come la risposta della pipeline. È passato per caso, perché era sulla porta giusta.

Adesso la cattura filtra sui campi che il datapath non riscrive, e IPv6 viene disabilitato sulle interfacce del banco.

3. La porta 0 era invisibile per costruzione

L'ingresso era anche la porta logica 0. Una classe che inoltrava su quella porta rimandava il frame fuori dall'interfaccia da cui era entrato — dove la cattura non lo distingue da quello appena iniettato. Una decisione *corretta* sulla porta 0 non era osservabile.

Il banco ha adesso un ingresso dedicato, separato da tutte le porte.

Un test che non può fallire non è un test. In tutti e tre i casi il risultato era verde e privo di contenuto, e in tutti e tre il difetto è emerso solo mettendo in mezzo un kernel vero.

6.4 Il risultato

Tutte e tre le pipeline superano oggi la verifica end-to-end su un datapath reale: XDP native, redirect vero, pacchetto ripreso sulla porta d'uscita, e scarto verificato dal *contatore* e non dal silenzio — perché "non è arrivato niente" è anche ciò che si osserva quando il redirect si perde.

La feature che non c'era

7.1 Il sintomo

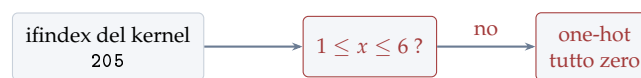
Il modello ha quattro ingressi. Uno di questi è *da quale interfaccia è arrivato il pacchetto*, codificato *one-hot*: un vettore di sei posizioni con un solo uno.

Il datapath leggeva l'indice dell'interfaccia dato dal kernel — l' *ifindex* — e lo usava direttamente come posizione nel vettore, con un controllo di validità:

$$1 \leq \text{ifindex} \leq n_{\text{interfacce}}.$$

Gli *ifindex* però li assegna il kernel, in ordine di creazione. Su un container appena avviato valgono 2, 3, 4. Su una macchina che ha creato qualche interfaccia virtuale valgono 205, 217, 229.

Il controllo falliva sempre. **Nessun bit veniva acceso.** Il modello girava di fatto su tre ingressi invece di quattro.



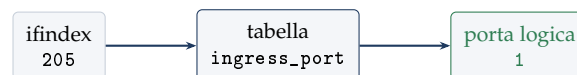
7.2 Perché nessun test lo vedeva

Perché nessun test controllava che quella feature *contribuisse* qualcosa. Riferimento e datapath la ignoravano entrambi, quindi **concordavano** — e l'accordo veniva registrato come successo.

Una pipeline aveva in effetti una tabella di traduzione, ma compilata dentro il programma come [2, 3, 4, 5, 6, 7]: la stessa assunzione, scritta in un altro modo. Le altre due usavano l'indice grezzo. Le tre pipeline sceglievano quindi *colonne diverse* per lo stesso pacchetto — una discrepanza documentata in un commento e mai risolta.

7.3 La correzione

La traduzione è diventata una **tabella a runtime**, riempita dal piano di controllo leggendo le interfacce del nodo:



È il **simmetrico** della tabella già usata in uscita: là "quale interfaccia realizza la porta che il modello ha scelto", qui "quale porta è l'interfaccia da cui il pacchetto è arrivato". Entrambe sono fatti del nodo, entrambe si risolvono quando il nodo esiste.

7.4 La prova che adesso è viva

Prima e dopo la correzione, gli ingressi che raggiungono ciascuna classe **cambiano**:

classe	prima (feature morta)	dopo (feature viva)
0	100110, ttl 27	100111, ttl 20
1	010000, ttl 2	010000, ttl 14
5	000000, ttl 2	000000, ttl 12

Stesso stato dei link, TTL diverso per ottenere la stessa classe: l'ingresso in più contribuisce. E il TTL ora sposta la decisione, cosa che prima non faceva.

7.5 Il prezzo, misurato

Una lettura di tabella in più per pacchetto:

	prima	dopo
letture per pacchetto (P1)	4,0	5,0
letture per pacchetto (P3)	26,0	28,0
istruzioni (P1)	997	971

Le istruzioni di P1 *calano*: la tabella ha sostituito un blocco switch a sei casi, e una lettura costa meno di un salto.

Una lettura in più arriverà poco dopo, per la stessa ragione applicata a un altro ingresso: è il capitolo che segue. Le cifre finali sono 6,0 letture per P1 e 29,0 per P3.

Il nodo non sapeva di essere un nodo

8.1 L'ingresso più grande di tutti

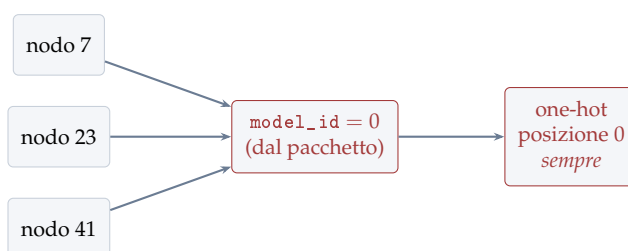
Dei quattro ingressi del modello, quello che pesa di più è *quale nodo sta elaborando il pacchetto*. Su Germany50 sono 52 nodi, quindi un vettore one-hot di 52 posizioni: **208 dei 319 pesi** della rete, quasi due terzi.

8.2 Il sintomo

Il datapath riempiva quel vettore con il campo `model_id` preso dall'intestazione IPA del pacchetto.

Sono due cose diverse. `model_id` dice *quale modello* il pacchetto chiede di usare; il vettore vuole sapere *quale nodo* lo sta elaborando. E il `model_id` non cambia lungo il percorso: è scritto dal mittente e viaggia identico fino alla destinazione.

Con un solo modello registrato — il caso normale — **ogni nodo della rete accendeva la stessa identica posizione**. Due terzi dei pesi della rete non distinguevano nulla.



8.3 La correzione, e perché una tabella e non un array

Stessa forma della correzione precedente: una **tabella letta a runtime**, riempita quando il nodo viene messo in servizio, dalla variabile d'ambiente `IPA_NODE_ID` o dall'indice del nodo nella topologia.

Un dettaglio che sembra tecnico e non lo è: la tabella è di tipo *hash* e non *array*. Un array in questo ambiente è preallocato e riempito di zeri, quindi una lettura riesce **sempre**: "non configurato" e "sono il nodo 0" tornerebbero lo stesso valore, indistinguibili. Sarebbe esattamente il difetto che la tabella serve a togliere. Con un hash, finché nessuno scrive non c'è voce: il vettore resta spento, ed è la risposta onesta.

Questo errore è stato commesso e corretto: la prima versione usava un array, e un test su un'architettura alternativa è fallito 5 volte su 5 prima che se ne capisse il motivo.

8.4 Una pipeline ha resistito un giorno

P1 e P2 hanno accettato la modifica subito. P3 no: il verificatore del kernel ha **rifiutato il programma**, dodici configurazioni di fila.

Il motivo è controintuitivo e vale la pena averlo scritto. Il verificatore non misura quanto è lungo il programma: esplora i *cammini* possibili, e il numero di stati che deve tenere è il **prodotto** dei valori indipendenti che segue dentro un ciclo srotolato, non la loro somma.

$$\text{stati} \sim v_1 \times v_2 \times \cdots \times v_k$$

L'unica versione che caricava era quella in cui l'indice del nodo *non era un valore indipendente*: `nodo = model_id`, cioè lo stesso registro che il verificatore stava già seguendo. Un valore, non due.

Escluse una a una, ognuna con la sua prova: il tipo di tabella, la sua dichiarazione, il valore, il tipo del dato, il numero di letture, la larghezza in bit. Nessuna era la causa.

Quello che ha liberato spazio non toccava il nodo: un **tetto** di tutt'altra feature. Il programma teneva in vita otto posizioni per un vettore di occupazione delle code che il modello depositato *non dichiara nemmeno*. Portato quel tetto da 8 a 1, il programma ha caricato — e più piccolo di prima.

Questi limiti sono **scogliere, non pendenze**. Nello stesso giro, ridurre *anche* un secondo tetto — cioè chiedere *meno* — ha fatto fallire di nuovo il caricamento. Un tetto qui si cambia e **si rimisura**; non si ragiona.

8.5 La prova che il nodo è vivo

Stesso programma compilato, due esecuzioni che differiscono solo per la tabella del nodo:

esecuzione	esito sul pacchetto
senza indice di nodo	scartato
indice di nodo = 7	non scartato

Nient'altro è cambiato fra le due. L'ingresso sposta la decisione.

Il verificatore non conta le istruzioni

9.1 Due monete diverse

Il tetto sulle code portato a 1 era una perdita vera: a 1, un modello che dichiara davvero quella feature viene *rifiutato*. Per recuperarlo serviva margine di verifica, e il margine è arrivato da una riscrittura che non toglie niente.

9.2 Il ciclo era girato dalla parte sbagliata

Il primo strato della rete si calcola così: per ogni neurone, si sommano i contributi di ogni feature dichiarata nel descrittore.

Il codice lo faceva nell'ordine ovvio — neuroni fuori, feature dentro:

```
per ogni neurone j (8 volte)
  per ogni feature f (4 volte)
    di che tipo e' f?      <- catena a 5 rami
    di che larghezza e' f? <- controllo per ogni posizione
  accumula
```

Ma *di che tipo* e *di che larghezza* è una feature non dipende dal neurone. Quella catena a cinque rami veniva quindi rivalutata $8 \times 4 = 32$ volte per pacchetto, e il controllo sulla larghezza di un vettore denso $8 \times 8 = 64$ volte, per ottenere ogni volta la stessa risposta.

Girato al contrario — feature fuori, neuroni dentro — diventa:

```
per ogni feature f (4 volte)
  di che tipo e' f?      <- 4 volte, non 32
  di che larghezza e' f? <- 8 volte, non 64
  per ogni neurone j (8 volte)
    accumula              <- nessun ramo qui dentro
```

9.3 Perché il risultato non cambia di un bit

Cambia solo l'**ordine in cui i termini vengono sommati**. I termini sono gli stessi, calcolati allo stesso modo. E la somma fra interi a 64 bit in complemento a due è associativa e commutativa — anche in caso di traboccamento, che si comporta identicamente nei due ordini:

$$(a + b) + c = a + (b + c).$$

Ogni accumulatore finisce quindi **bit per bit identico**. Non "circa uguale": identico.

9.4 Il risultato, che sembra un errore

	prima	dopo
P2, programma principale	16 144	14 587
P3, primo strato	6 417	6 850
P3, tetto sulle code caricabile	1	8

P3 è **cresciuto** di 433 istruzioni ed è diventato molto più economico da verificare. Prima della riscrittura, col tetto delle code a 8 non caricava niente; dopo, carica a 10 207 istruzioni.

Dimensione del programma e complessità di verifica sono **monete diverse**. Il verificatore paga i cammini.

9.5 Un effetto collaterale che vale una riga di tabella

Contare quante letture di tabella fa un programma per pacchetto richiede di ricompilarlo con un contatore su ogni punto di lettura. Le versioni strumentate di P2 e P3 **sforavano il verificatore**: per questo la riga "letture di tabella" riportava *n.d.* per entrambe.

Con il margine liberato dalla riscrittura, tutte e due caricano. La misura che era impossibile è diventata una riga di tabella — 11 letture per P2, 29 per P3 — senza che nessuno abbia riscritto il contatore.

I numeri, e che cosa dicono davvero

10.1 La tabella

Misure su una macchina virtuale a 4 processori, modello 65-4-4-7, scala 24, `test_suite --only kernel` del 23 settembre; P1 è l'oggetto AOT.

Rimisurato il 23 settembre

Le tabelle e i grafici di questo capitolo vengono dalla rimisura del 23 settembre, con P1 e P1.5 costruite come **oggetto precompilato** (AOT), cioè come vanno sui nodi. In quella sessione la macchina virtuale era più veloce che nella precedente: *tutte* le latenze sono scese di circa il 35–40%, anche quelle di P2 e P3, il cui codice non è cambiato. È lo stato dell'host (processore ibrido, vedi più sotto), non un miglioramento dei programmi. I **rapporti** fra le pipeline non si sono mossi — P2 costa circa 4,5 volte P1 prima e dopo — e le istruzioni sono deterministiche. Le cifre assolute di sessioni diverse non vanno confrontate fra loro.

	baseline	P1	P2	P3
istruzioni caricate	155	1 011	14 985	12 349
salti fra programmi	0	1	1	3
letture di tabella	3,0	6,0	11,0	29,0
memoria delle tabelle	280 B	2 516 B	10 416 B	19 508 B
latenza (minimo di 7)	17 ns	50 ns	180 ns	278 ns

Il *baseline* è un programma che analizza il pacchetto e lo rimanda fuori senza alcuna inferenza: è il pavimento contro cui si legge tutto il resto.

Tutte le colonne vengono dalla **stessa esecuzione**, sulla stessa macchina, nello stesso minuto. La riga "letture di tabella" è piena per la prima volta: fino alla riscrittura del capitolo precedente, per P2 e P3 diceva *n.d.*

10.2 Primo risultato: la dimensione non predice la velocità

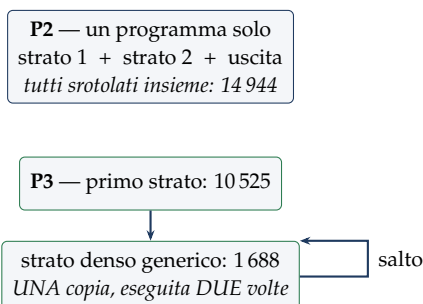
P3 ha il 18% di istruzioni in meno di P2 ed è il 54% più lento.

Il motivo sta nelle due righe centrali: tre salti fra programmi invece di uno, e ventinove letture di tabella per pacchetto invece di undici.

Il conteggio delle istruzioni misura quanto è *grande* il programma caricato, non quanto *lavoro* fa per pacchetto. In questo regime a dominare sono le letture di memoria e i salti, non l'aritmetica.

Perché P2 è più grande, pur essendo più veloce

Le due cose hanno la stessa causa: **P3 non tiene la rete intera in un programma.**



P3 srotola **un** solo strato denso generico e ci rientra, con un salto fra programmi, a ogni passo della rete. Così la *profondità* del modello non costa dimensione: un modello a quattro strati gira sullo stesso identico binario. P2 deve srotolarli tutti, perché è un programma solo — e un modello più profondo richiede un altro binario.

Quel riuso è esattamente ciò che P3 paga in latenza: ogni rientro è un salto più le letture che servono a ricostruire il contesto perduto.

La stessa scelta di progetto spiega *entrambi* i numeri: P3 è più piccolo **perché** riusa un pezzo, ed è più lento **perché** riusarlo costa un salto e delle letture ogni volta.

10.3 Secondo risultato: quanto valgono le latenze assolute

Ogni cifra di latenza in tabella è il **minimo di sette misure** indipendenti. Guardando come sono distribuite quelle sette, dentro una singola esecuzione:

	baseline	P1	P2	P3
minimo	17 ns	50 ns	180 ns	278 ns
mediana	20 ns	56 ns	233 ns	294 ns
massimo	35 ns	58 ns	348 ns	336 ns
dispersione	106%	16%	93%	21%

Il 93% di P2 è *dentro* una sola esecuzione, sullo stesso binario, in pochi secondi (e la baseline arriva al 106% su numeri piccoli; nella misura precedente era P1 a disperdersi di più, al 36%: chi esce peggio cambia da un'esecuzione all'altra). Le **istruzioni sono identiche** fra esecuzioni diverse: sono deterministiche. Le latenze no.

Su questa macchina virtuale una differenza di latenza sotto il 20%–30% **non significa nulla**, nemmeno dentro la stessa esecuzione. Valgono i rapporti fra le colonne, non le cifre.
Ogni valore assoluto va letto come ordine di grandezza su questa VM, mai come prestazione del sistema.

Un caso in cui la regola è servita

Il tetto sulle code di P3, riportato da 1 a 8, aggiunge **3 357 istruzioni** al programma:

	tetto 1	tetto 8
istruzioni (P3)	8 674	12 031
letture per pacchetto	29,0	29,0
latenza	419 ns	421 ns

Il 39% di istruzioni in più costa **2 nanosecondi**, cioè niente, e le letture non si muovono di una. Il motivo è semplice: il modello depositato non dichiara la feature delle code, quindi quel ramo del programma *non viene mai eseguito*. Cresce la dimensione statica; il percorso eseguito è lo stesso.

Una metrica che si sposta del 39% senza che cambi nulla di ciò che gira **non è una misura di costo**. Il numero di istruzioni va letto come "quanto è grande il binario", mai come "quanto lavora".

Questo ha reso necessario rifare la tabella dei risultati: la versione precedente presentava tre esecuzioni etichettate come ripetizioni, ma venivano in realtà da *stati diversi del codice*.

10.4 Terzo risultato: la genericità ha un costo di verifica

P2 serve qualsiasi modello che stia sotto certi tetti dichiarati a priori (numero massimo di neuroni, di ingressi, di feature). È il suo pregio: non ricompila mai.

Il conto però lo paga il verificatore del kernel, che percorre ogni cammino del corpo srotolato. Con i tetti larghi il kernel si arrende:

```
processed 1000001 insns (limit 1000000) ... total_states 13864
```

Da notare: il programma è lungo circa 14 900 istruzioni, **ampiamente dentro** il limite di dimensione. Quello che finisce è la *complessità di verifica*, che è un limite diverso e cresce col **prodotto** dei tetti:

$$\text{costo di verifica} \sim H_{\max} \times F_{\max} \times N_{\max}.$$

È la misura concreta di quanto un programma "generico" possa esserlo. La genericità di P2 non è gratis: si paga in stati del verificatore, e il tetto si tocca con numeri ragionevoli (8 neuroni, 4 feature, 128 ingressi).

10.5 Quarto risultato: togliere il compilatore dal nodo

P1 scrive i pesi *dentro* il codice come numeri letterali. È ciò che la rende veloce — il compilatore può far sparire le moltiplicazioni per zero e trasformare in scorrimenti quelle per potenze di due — ma ha un prezzo: ogni volta che il modello cambia, bisogna **rieseguire il compilatore sul nodo**.

Misurato: **741 ms** nella sessione più recente, 1 242 ms in una precedente. In quel tempo il nodo non sta aggiornando niente.

L'alternativa è compilare *prima*, su una macchina qualunque, e spedire al nodo l'oggetto già pronto. Al momento del deploy il nodo apre il file e lo carica, senza compilatore:

	con compilatore sul nodo	oggetto precompilato
costo di messa in servizio	741 ms	9–10 ms
istruzioni	1 026	1 011
latenza per pacchetto	44 ns	46 ns

Quasi due ordini di grandezza sul costo di aggiornamento, e **nessun costo per pacchetto**: 46 ns contro 44 ns è la stessa cifra, dentro il rumore misurato sopra. La riduzione fatta dal compilatore sui pesi letterali resta dentro l'oggetto.

Una cautela sui numeri. La versione precedente di questa tabella riportava 74 contro 73 ns: il 74 misurava un altro percorso (il pacchetto scadeva dopo poche decine di esecuzioni, vedi capitolo 14). E il costo di caricamento varia fra esecuzioni — sono stati osservati da 4,7 a 37,2 ms per lo stesso oggetto. Quello che regge è *l'ordine di grandezza* rispetto al compilatore, non la cifra precisa.

Vale solo quando i modelli sono noti in anticipo, che è l'ipotesi sotto cui P1 esiste.

10.6 Che cosa questi numeri non dicono

BPF_PROG_TEST_RUN esegue il programma in un ciclo sullo stesso buffer. Niente scheda di rete, niente driver, nessuna allocazione, nessuna pressione di cache da traffico vero.

Il valore in "milioni di pacchetti al secondo" è semplicemente $1/\text{latenza}$: un **picco teorico**, non un throughput retto. Per quello serve traffico vero, generato a ritmo, e la misura di quanti pacchetti si perdono.

Come cresce il costo

11.1 La domanda

Fin qui tutto è misurato su *un* modello e *una* topologia. La domanda che manca è come cambia il costo quando la rete o il modello diventano più grandi — e soprattutto se le tre pipeline hanno **pendenze diverse**.

Se le pendenze fossero uguali, averne più di una sarebbe un esercizio. Non lo sono, ed è questo il capitolo che lo mostra.

Le pipeline confrontate sono **quattro**, e formano una scala di *quanto si sa quando il programma viene compilato*:

	compilato dentro	un binario per
P1 specializzata	pesi e indice di questo nodo	nodo
P1.5 hardcoded	pesi; il nodo si legge a runtime	modello
P2 template	solo i soffitti	tutto
P3 modular	anche la profondità è a runtime	tutto

La P1.5 è quella che i capitoli precedenti chiamano semplicemente P1: la *disposizione* delle feature è scritta nel codice, ma l'indice del nodo no. La P1 specializzata congela anche quello — ed è il confronto con cui si chiude il capitolo.

11.2 Come è impostata la misura

Cinque assi, e su ciascuno si muove **una sola** variabile: tutto il resto resta fermo, così una curva si può attribuire a quello che c'è scritto sull'asse delle ascisse e a nient'altro.

asse	valori	si ferma perché
nodi della rete	10, 25, 52, 75, 100	il tetto sugli ingressi è 128
hidden layer	da 1 a 6	—
neuroni per layer	2, 4, 6, 8	il tetto compilato è 8
composizione dell'IV	4 varianti	0, 1 o 2 one-hot
pesi a zero	0% ... 90%	—

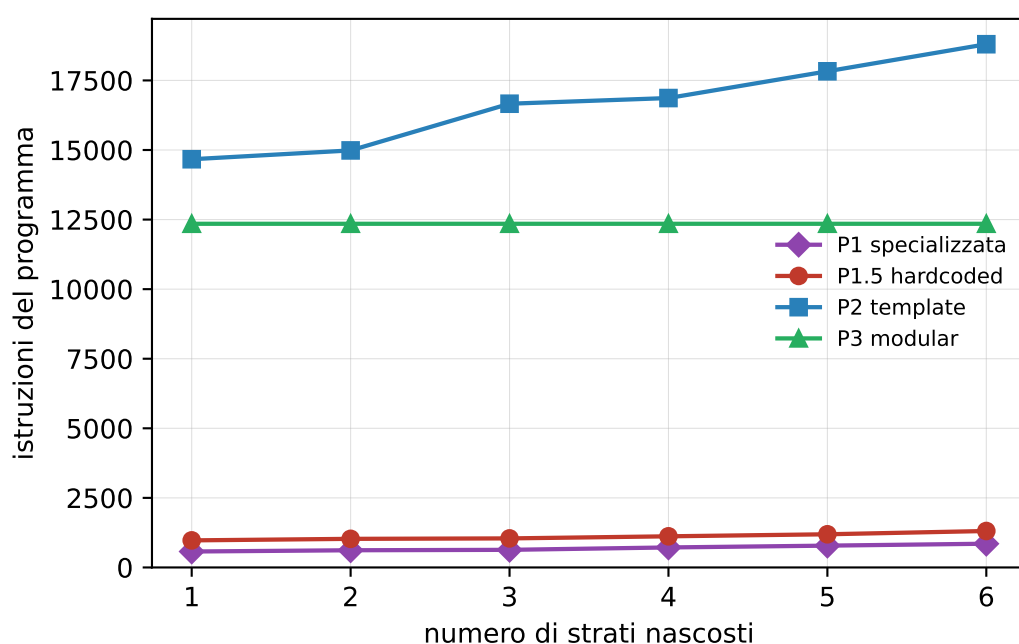
Tre accortezze che hanno cambiato i risultati, non solo la forma:

- **Ogni misura in un processo separato.** P1 può far abortire il compilatore in modo non recuperabile; e un rifiuto del verificatore è un *dato*, non un guasto dello strumento.
- **I pesi vengono da un sacchetto fisso**, e un modello più grande *estende* quello più piccolo invece di sostituirne tutti i pesi. Senza questo, spostandosi lungo un asse cambiavano due cose insieme, e parte della curva di P1 era rumore dei pesi travestito da asse x.
- **Un allarme automatico sulle misure sporche:** se una pipeline ha lo stesso identico programma e la latenza varia più del doppio lungo l'asse, lo scarto è la macchina e lo strumento lo dice.

11.3 Profondità: chi paga in spazio e chi in tempo

Rimisurato il 23 settembre

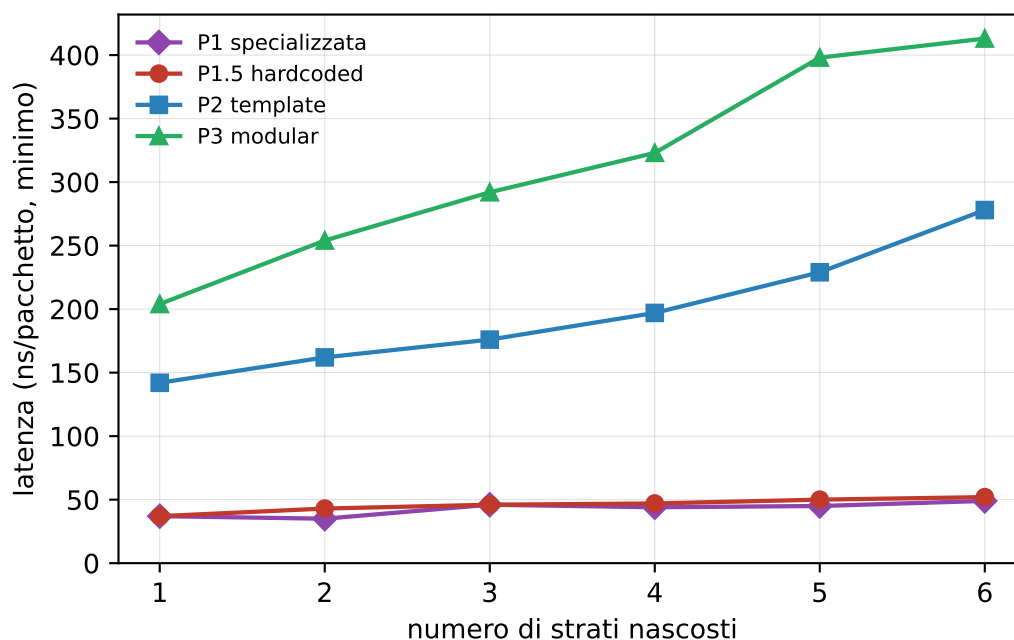
Le tabelle e i grafici di questo capitolo vengono dalla rimisura del 23 settembre, con P1 e P1.5 costruite come **oggetto precompilato** (AOT), cioè come vanno sui nodi. In quella sessione la macchina virtuale era più veloce che nella precedente: *tutte* le latenze sono scese di circa il 35–40%, anche quelle di P2 e P3, il cui codice non è cambiato. È lo stato dell’host (processore ibrido, vedi più sotto), non un miglioramento dei programmi. I **rapporti** fra le pipeline non si sono mossi — P2 costa circa 4,5 volte P1 prima e dopo — e le istruzioni sono deterministiche. Le cifre assolute di sessioni diverse non vanno confrontate fra loro.



P2 cresce di circa 830 istruzioni per ogni hidden layer. P3 di zero.

P3 srotola *un* solo strato denso generico e ci rientra a ogni passo della rete, quindi la profondità non entra nel programma: la sua riga è perfettamente orizzontale, da uno a sei strati. P2 deve srotolarli tutti, perché è un programma solo.

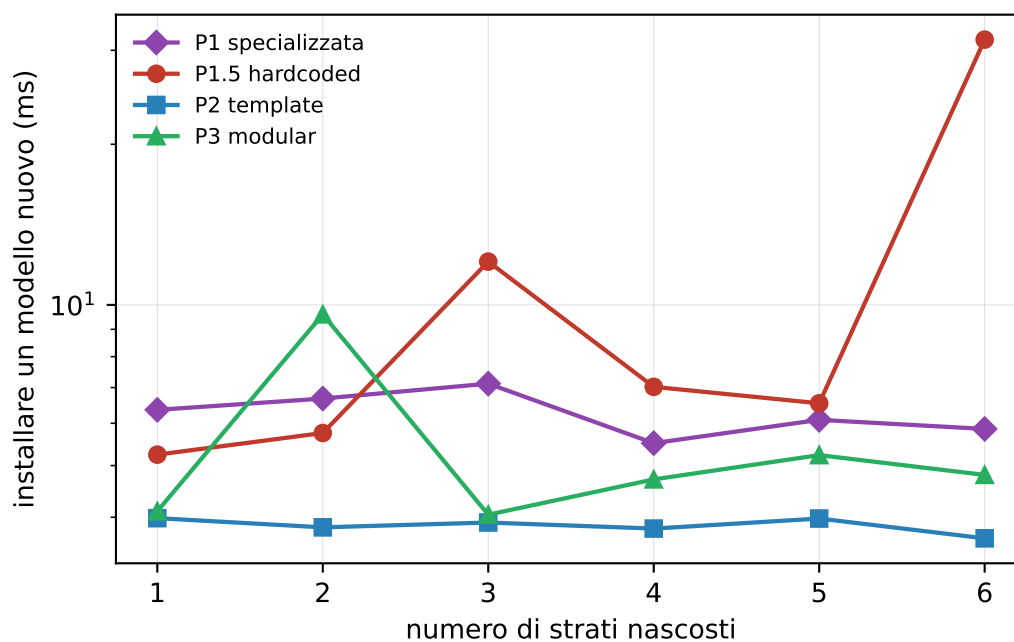
Ma la storia non finisce lì:



E P3 lo paga in tempo: circa 42 nanosecondi per strato, contro i 27 di P2 (204 → 413 ns da uno a sei strati). Sono i salti fra programmi, da 2 a 7.

Le due figure hanno la *stessa* causa vista dai due lati. P3 è più piccolo **perché** riusa un pezzo, ed è più lento **perché** riusarlo costa un salto e delle letture ogni volta. È il compromesso della modularità, ridotto a due numeri.

11.4 Aggiornare il modello: con l'oggetto precompilato, pari



Attenzione alla scala verticale: è **logaritmica**, e qui ingrandisce il rumore. Tutte e quattro le curve stanno fra 4 e 31 millisecondi; i picchi (P1.5 a sei strati, P3 a due) sono singole esecuzioni del caricamento, che varia molto da una volta all'altra. Nella misura precedente P1 stava sopra il secondo e le altre due a pochi millisecondi: la scala logaritmica serviva a vederle tutte.

Con P1 costruita come oggetto precompilato, installare un modello nuovo su un nodo già in servizio costa **5–8 millisecondi** in P1 — il nodo apre e carica un file pronto — e **4–10 millisecondi** in P2 e P3, che scrivono in una tabella. **Sono pari.**

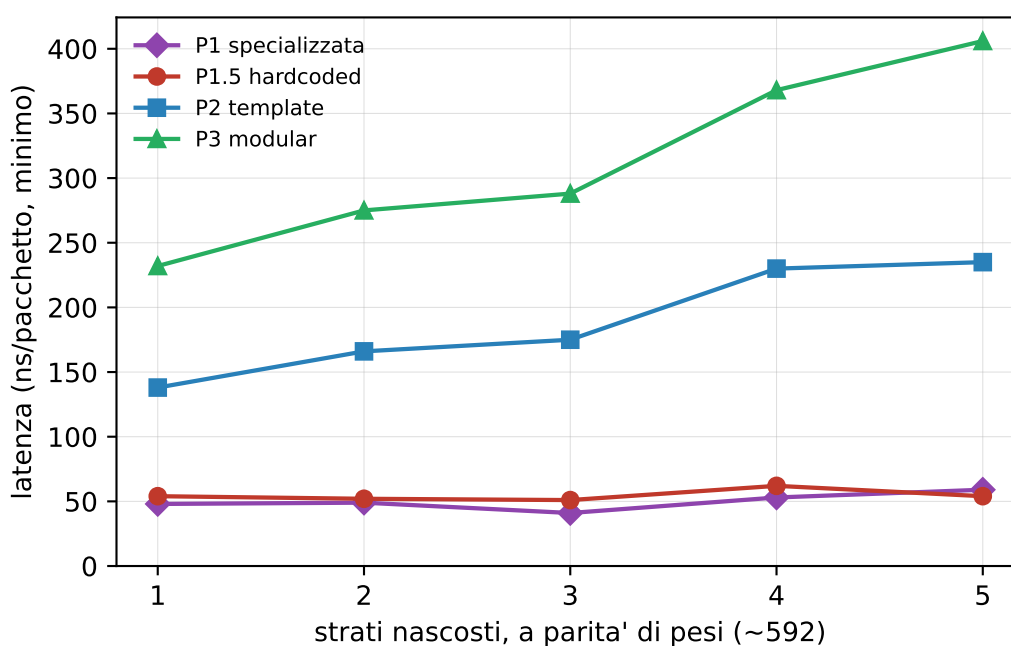
Anche contando tutto il lavoro di P1 fuori dal nodo — generare il C, compilarlo con clang, caricarlo — si sta fra 70 e 140 ms. Il «circa 1,4 secondi» della misura precedente non era il compilatore: era **BCC**, che a ogni compilazione analizza gli header del kernel. P2 e P3 pagano la stessa cosa una volta sola, all'avvio del nodo (1,3–1,8 s), e mai più.

La differenza che resta fra le pipeline non è quindi il tempo di aggiornamento, ma *dove* serve il compilatore: P1 ne vuole uno per ogni modello nuovo, su qualche macchina; P2 e P3 no.

11.5 Larga o profonda? Dipende da chi la esegue

Dato un numero di parametri da spendere, conviene metterli in *un* layer largo o in *molte* stretti? È una domanda che si fa di solito parlando del modello, come se la risposta fosse una proprietà della rete neurale. Qui non lo è.

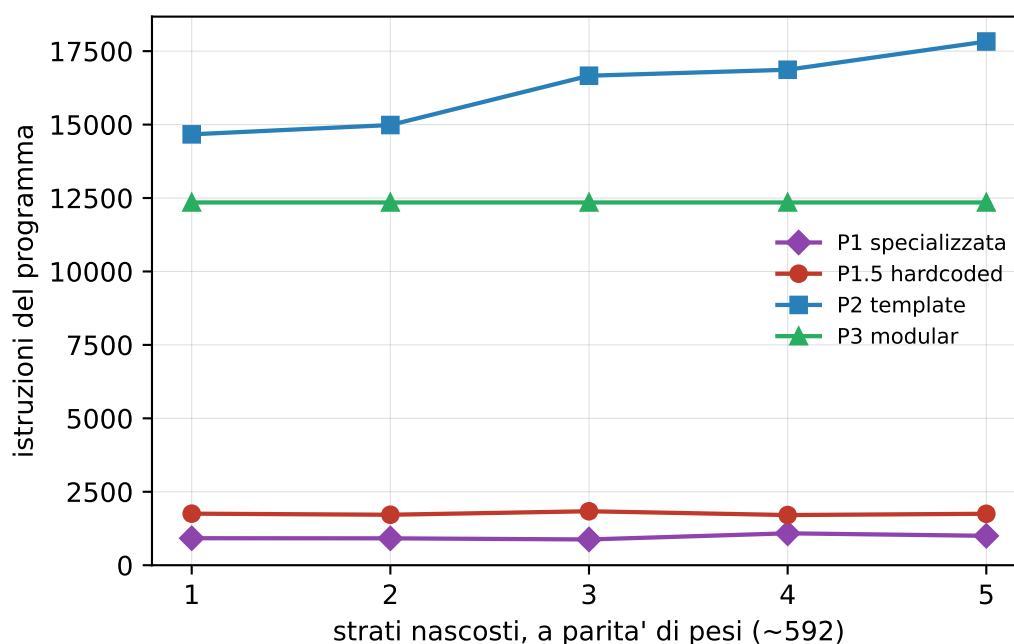
Cinque forme, tutte con lo stesso budget — 592 pesi, con uno scarto del 2% — disposto in modo diverso: da un solo strato da otto neuroni a cinque strati stretti. Tutte stanno dentro i soffitti compilati, quindi tutte e quattro le pipeline possono eseguirle tutte.



A parità di parametri, le due versioni hardcoded **non distinguono** una rete larga da una profonda (41–62 ns su tutte e cinque le forme). Da uno a cinque strati P2 perde il **70%** di latenza (138 → 235 ns) e P3 il **75%** (232 → 406 ns).

La stessa scelta di architettura costa zero o costa il doppio, a seconda della pipeline su cui finisce.

E i due che perdono, perdono per ragioni diverse:



P2 cresce anche in dimensione (da 14 668 a 17 828): deve srotolare ogni strato, perché è un programma solo. P3 ha la riga **perfettamente orizzontale** — 12 349 istruzioni a tutti e cinque i punti — e perde comunque il 75% di velocità.

Dimensione costante e tempo crescente è la firma di un costo di transizione, non di calcolo. In P3 si vede allo stato puro: ciò che cresce non è il lavoro, è il numero di volte che il programma deve rientrare in sé stesso.

E la conclusione di prima?

Un capitolo fa avevamo scritto che allargare batte approfondire. Qui le due P1 dicono di no. Non è una contraddizione: **dipende dal budget**.

(misura precedente, <i>bench_depth_vs_width</i> : non rifatta il 23/09)			
budget	larga	profonda (8 strati)	
~300 pesi	54 ns	69 ns	(+28%)
~1 200 pesi	124 ns	282 ns	(+127%)

A 592 pesi e cinque strati l'effetto su P1 sta sotto il rumore della misura; a 1 200 pesi e otto strati è più che doppio.

E c'è una seconda condizione, emersa provando quattro vettori d'ingresso diversi a budget abbinato. Con un ingresso **grande e dominato da una one-hot** — il caso di IPA, 52 dei 65 valori — allargare batte approfondire nettamente: +140% di latenza a otto strati. Con un ingresso **piccolo e denso** (undici valori, nessuna one-hot) il vantaggio **sparisce**: a 300 pesi la versione a otto strati è perfino un poco più veloce, e più piccola.

Il motivo è lo stesso di due sezioni fa. Una one-hot larga costa poco per pacchetto, perché il datapath ne legge *una colonna sola*; un ingresso denso invece fa moltiplicare le letture per la larghezza del primo strato.

La frase giusta non è "le reti larghe sono preferibili a quelle profonde". È: *con un ingresso dominato da feature one-hot, allargare batte approfondire, e sempre di più al crescere del budget; con un ingresso piccolo e denso il vantaggio sparisce, e oltre una certa taglia è la forma larga a non compilare più.*

La prima è una legge, e i dati non la sostengono. La seconda è un risultato con un dominio di validità.

Il limite mangia prima le larghe

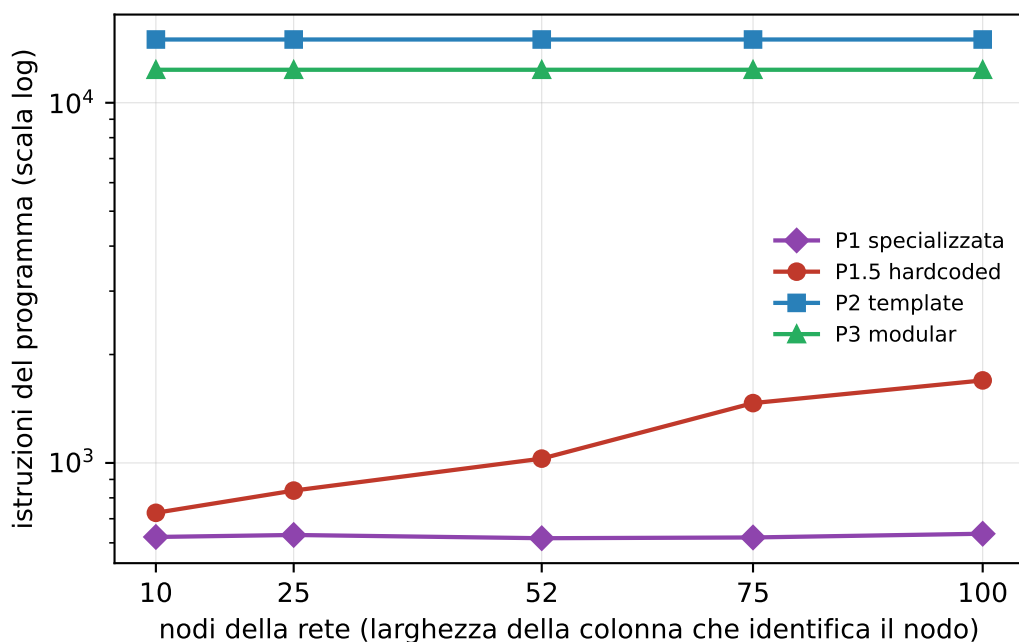
A parità di parametri la forma larga sfonda lo stack eBPF da 512 byte *prima* di quella profonda, perché è il primo strato a consumarlo. Su undici ingressi e 1 200 pesi, in ordine di stack stimato: larga 1×63 crash, 2×26 crash, profonda 4×17 carica, profonda 8×11 carica.

Con un ingresso piccolo e un budget medio, **la rete profonda è l'unica che sta in piedi.**

11.6 Congelare il nodo: conviene?

L'indice del nodo è una **costante del deployment**: un nodo non cambia di posto. Il codice però lo legge a runtime, e per farlo porta uno switch con un caso per ogni nodo della rete — 52 casi per Germany50, di cui ne esegue *uno*.

Congelandolo a tempo di generazione lo switch sparisce, e con lui la lettura della tabella. Restano quattro numeri costanti, che il compilatore può piegare dentro al conto.



La taglia della rete entra nel programma **solo** nella P1.5, che passa da 727 a 1695 istruzioni fra 10 e 100 nodi.

Congelando il nodo quella dipendenza **sparisce**: la riga viola è piatta, fra 618 e 636 a qualunque taglia.

(La scala è logaritmica: in lineare le due righe P1 resterebbero schiacciate contro le quattordicimila di P2, e il confronto fra loro — che è il punto della figura — non si vedrebbe.)

Ma la velocità non cambia

nodi	10	25	52	75	100
P1 specializzata	40	39	37	32	37 ns
P1.5 hardcoded	39	43	48	38	44 ns

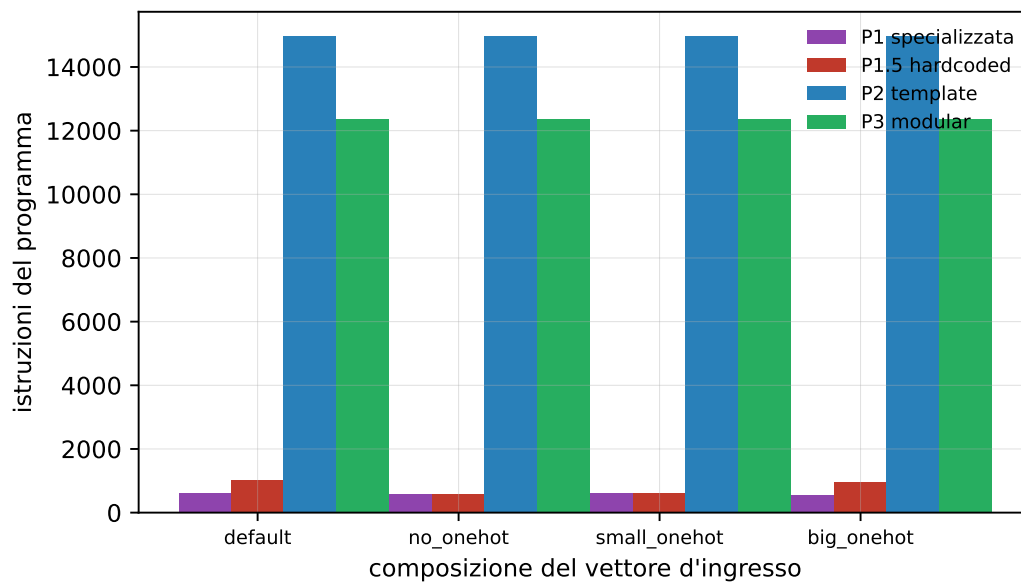
Praticamente identiche, dentro il rumore.

Il motivo per cui la *dimensione* crolla e il *tempo* no è lo stesso: lo switch ha 52 casi ma ne esegue uno. A runtime è un salto indicizzato, che costa poco; quelle 208 assegnazioni sono compilate per eseguirne quattro.

Toglierle libera molto **spazio** e quasi nessun **tempo**.

Il controllo, e perché ci si può fidare

Come si fa a essere sicuri che il divario venga davvero dalla one-hot del nodo e non da qualcos'altro che è cambiato per caso? Si prende un vettore d'ingresso che la feature *non* la contiene:



Nelle due composizioni senza la feature node le due P1 sono **identiche alla cifra** (596 e 596; 605 e 605). In quelle che ce l'hanno, divergono. Il divario è tutto lì e nient'altro.

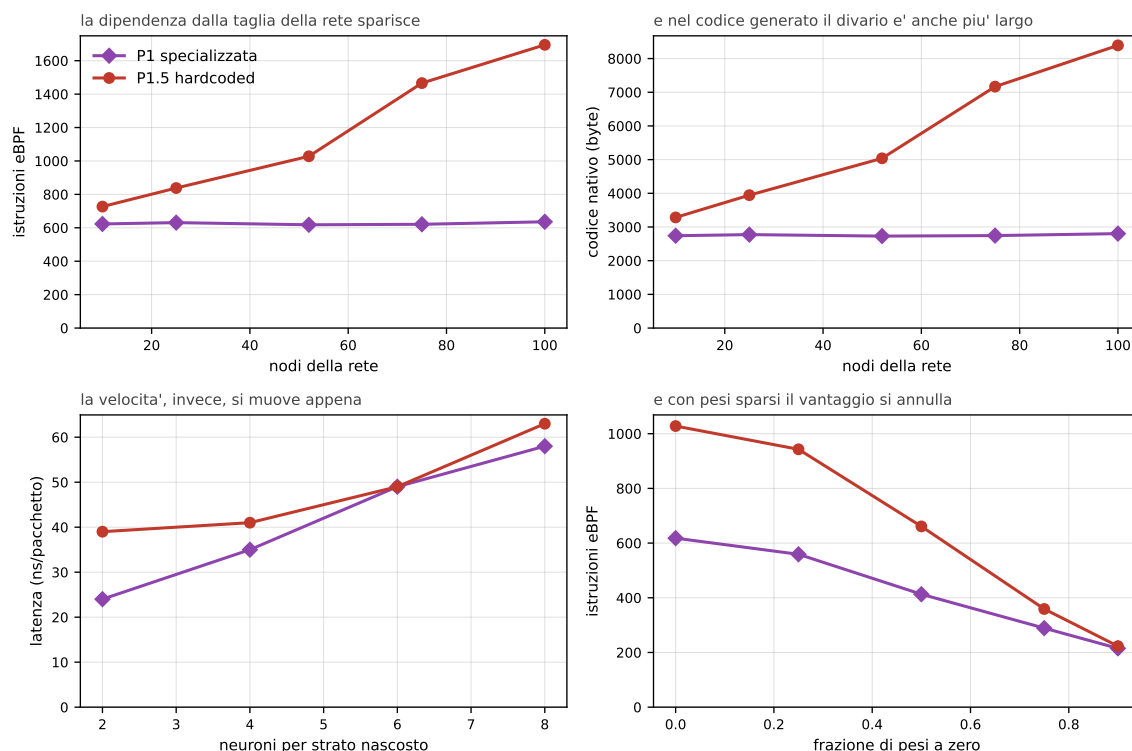
C'è poi una verifica che viene prima di ogni numero. Congelare il nodo significa scegliere *una colonna* della matrice dei pesi al momento di generare il codice: sceglierne una sbagliata darebbe un programma più piccolo, più veloce, e che calcola **un altro modello** — e nessuna misura di costo se ne accorgerebbe. Ottanta casi, dieci valori di TTL per otto configurazioni di link: la versione congelata decide sempre la stessa classe di quella che legge il nodo dalla tabella.

I due vantaggi non si sommano

pesi a zero	0%	25%	50%	75%	90%
P1 specializzata	618	559	413	289	215
P1.5 hardcoded	1 028	943	661	359	223

Al 90% di zeri il divario è sparito. Sparsità e nodo congelato sono **due strade alla stessa riduzione**: se i pesi sono già quasi tutti zero, il compilatore cancella lo switch da solo e non resta niente da congelare.

Le quattro risposte in una figura



Qui ci sono solo le due P1: in tutte le altre figure P2 e P3 stanno un ordine di grandezza più su e schiacciano questo confronto sul fondo del grafico.

I quattro pannelli non dicono la stessa cosa, ed è questo il punto. La dimensione crolla e smette di dipendere dalla rete; il codice nativo crolla ancora di più (2803 byte contro 8392 a cento nodi); la velocità si muove appena; e con i pesi sparsi le due curve si toccano.

Due aspettative smentite

su Germany50	specializzata	P1.5
istruzioni eBPF	618	1 028
codice nativo (byte)	2 730	5 039
latenza	37 ns	48 ns
memoria delle tabelle	2516 B	2516 B
aggiornare il modello sul nodo	6 ms	5 ms
generare, compilare e caricare	68 ms	84 ms

Le ultime due righe meritano attenzione perché smentiscono due aspettative ragionevoli.

La **memoria delle tabelle non cambia**: il programma specializzato non *legge* più la tabella del nodo, ma continua a *dichiararla*, perché l'intestazione è condivisa. Sono pochi byte, ma è codice morto.

Il **costo di aggiornamento non cala**: sul nodo le due caricano un oggetto pronto in pochi millisecondi, e fuori dal nodo la compilazione costa circa uguale (68 contro 84 ms). La specializzazione non rende la compilazione più veloce — e siccome ne servono N invece di una, il costo totale di installazione *peggiora* di un fattore N.

Il verdetto

Congelare il nodo **non è un'ottimizzazione di velocità**. È ciò che rende la pipeline hardcoded *praticabile su una rete grande*.

Lo switch della P1.5 cresce col numero di nodi moltiplicato per la larghezza del primo strato. Su una topologia da 500 nodi con 8 neuroni sarebbero quattromila assegnazioni srotolate, e c'è una taglia oltre la quale quel programma non si carica più. La specializzata non ci arriva mai.

Il prezzo è **un binario per nodo**: 52 compilazioni e 52 installazioni, una per macchina. Il costo di aggiornamento si moltiplica proprio per la grandezza da cui la specializzazione ha appena liberato la dimensione.

Su Germany50, dove la differenza è fra 618 e 1 028 istruzioni a velocità quasi uguale, è una scelta legittima in entrambe le direzioni. Su una rete dieci volte più grande non lo è più.

11.7 Congelare quali porte esistono

La sezione precedente congela *dove* il nodo si trova. Questa congela *quante interfacce ha*, che è una cosa diversa e si presta a un equivoco che vale la pena togliere di mezzo subito.

Il modello riserva sei colonne per `link_state` perché sei è il grado del nodo **più connesso** della rete. Ma su Germany50 i gradi vanno da 2 a 5, la media è 3,52, e **nessun nodo arriva a sei**. Su un nodo di grado 3 le tre colonne restanti non sono link caduti: dietro non c'è alcuna interfaccia. Valgono zero adesso, varranno zero sempre.

Non si congela il **valore** della feature, si congela la sua **struttura**.

Se un link sia su o giù resta letto a runtime, esattamente come prima. Quello che diventa noto alla compilazione è *quali posizioni possono esistere*.

Il generatore prende l'insieme delle porte presenti e non emette affatto le colonne che restano fuori. Il termine che sparisce è `link_state[i] · w[j][i]` con `link_state[i]` nullo: la somma è fra interi a 64 bit, quindi togliere un addendo nullo non cambia un bit. Gli offset dei pesi non si spostano, l'ordine delle feature non cambia, n_{in} resta quello del modello.

Quanto costa una colonna che non serve

porte presenti	2	3	4	5	6
istruzioni eBPF	549	567	584	600	618
latenza	33	31	32	34	38 ns

Le istruzioni scendono in modo lineare e pulito: gli scarti sono +18, +17, +16, +18, cioè **17 istruzioni per porta**. Un nodo di grado 2 porta l'**11,2 %** di programma in meno di uno di grado 6.

La latenza non si muove in modo leggibile: l'escursione è del 23 % su un valore intorno ai 35 ns.

Fra grado 6 e grado 2 spariscono **sedici** moltiplicazioni-accumulo, che a 3,2 GHz valgono all'incirca 5 ns. Il rumore di questo banco è il doppio. L'effetto c'è nel codice ed è sotto la risoluzione della misura: la riduzione di dimensione è dimostrata, il guadagno di tempo **no**.

Un'aspettativa smentita

Mi aspettavo il contrario, e vale la pena dire perché. Nel caso del nodo il tempo non cambia per un motivo chiaro: lo switch ha 52 casi ma ne esegue *uno*, quindi quelle 208 assegnazioni sono codice mai eseguito. `link_state` è invece un vettore **denso**: quelle moltiplicazioni vengono eseguite davvero, a ogni pacchetto. Sembrava dovesse seguirne un guadagno visibile.

Il ragionamento è corretto ma la conclusione no: sono sedici operazioni su circa 980 istruzioni per pacchetto. Il meccanismo è reale, la sua ampiezza è trascurabile.

Il controllo, che è servito davvero

Il confronto è fra due programmi che differiscono *solo* per questo: il nodo è congelato in entrambi. Ottanta casi — dieci valori di TTL per otto configurazioni di link realizzabili — danno la stessa classe in tutti.

Ma quel numero, da solo, non dimostra niente. Se quelle colonne non spostassero mai la decisione, «le due versioni concordano» sarebbe vero anche per una specializzazione **sbagliata**. Serve un controllo al contrario: accendere uno slot che sul nodo non esiste, e pretendere che a quel punto le due divergano.

Al primo tentativo quel controllo è rimasto **muto**. Non per un difetto del codice: una colonna di `link_state` contribuisce $1 \cdot w$ a un accumulatore che ne somma 65, e su quei pesi non ribaltava mai l'argmax. È una proprietà dei *pesi*, non della specializzazione. Cercando fra più semi del generatore di pesi il controllo morde — 29 divergenze — e solo a quel punto l'ottanta su ottanta diventa una prova.

Un test che passa anche quando la sua ipotesi è violata non sta verificando l'ipotesi. Il primo run era **inconcludente**, non positivo, e il banco adesso fallisce invece di annottarlo in giallo.

Il controllo sull'asse

Le altre tre pipeline non specializzano nulla, quindi lungo questo asse devono restare ferme — e lo sono: 1 028, 14 985 e 12 349 istruzioni su tutti e cinque i punti, con 319 pesi ovunque. L'asse di per sé non costa niente, quindi la pendenza della riga specializzata viene da lì e da nient'altro.

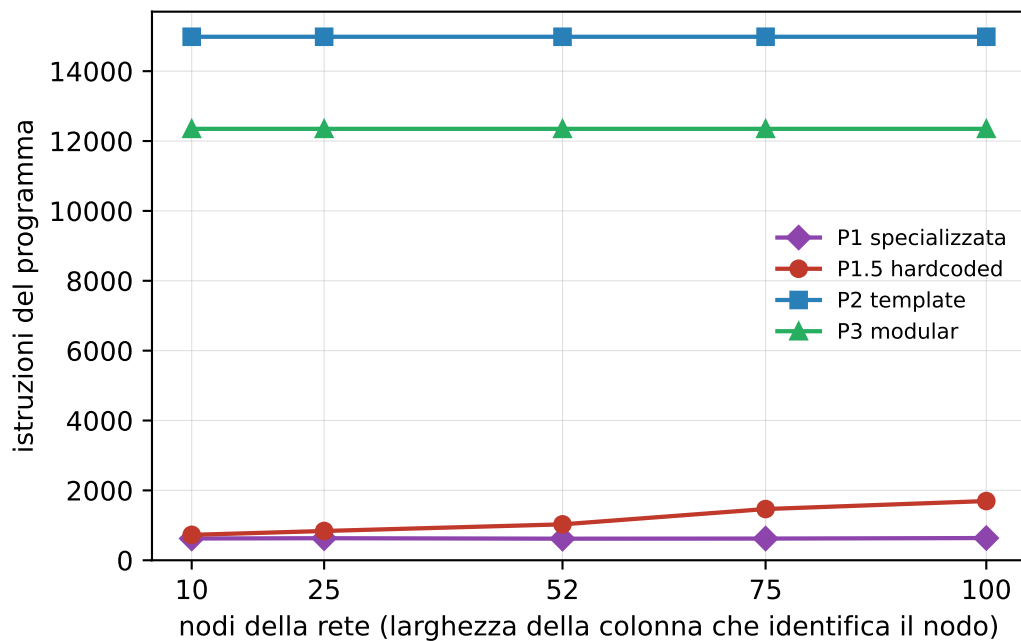
Vale anche il contrario: a sei porte il programma generato è **identico byte per byte** a quello non specializzato. Le 410 istruzioni di distacco che a Germany50 separano le due P1 sono quindi interamente la one-hot del nodo, e niente affatto le porte.

Quello che resta da misurare

L'incrocio con la sparsità. Con pesi quasi tutti nulli il compilatore cancella già per conto suo parte di quelle moltiplicazioni, e i due effetti potrebbero non sommarsi — è esattamente quello che succede fra nodo congelato e sparsità, dove al 90 % di zeri il divario sparisce.

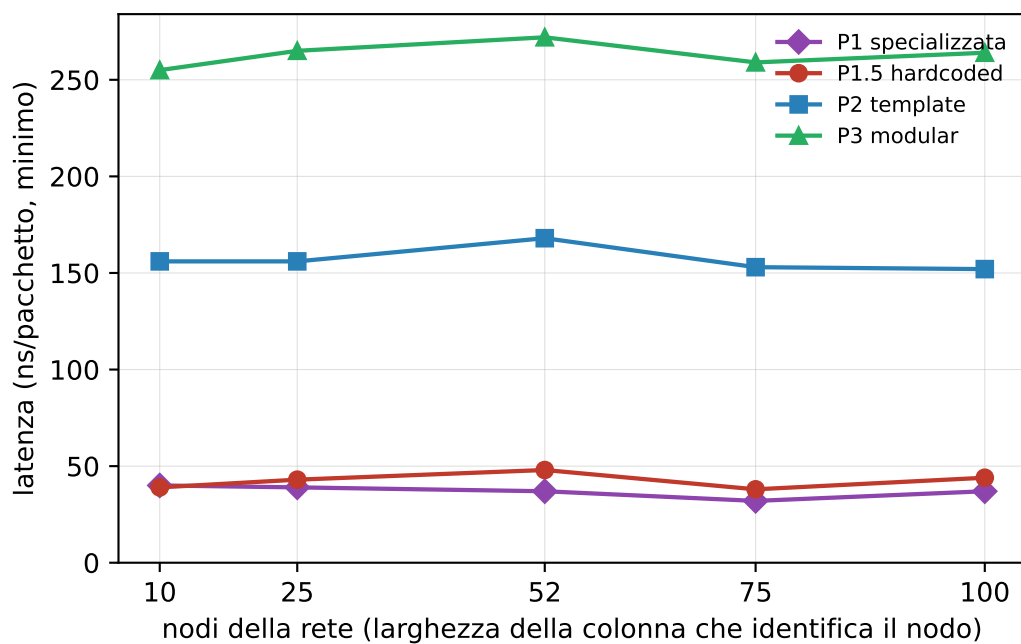
11.8 Due larghezze, due risposte opposte

Questa coppia di figure è il risultato che preferisco di tutta l'analisi, perché la risposta non è quella che verrebbe da indovinare.



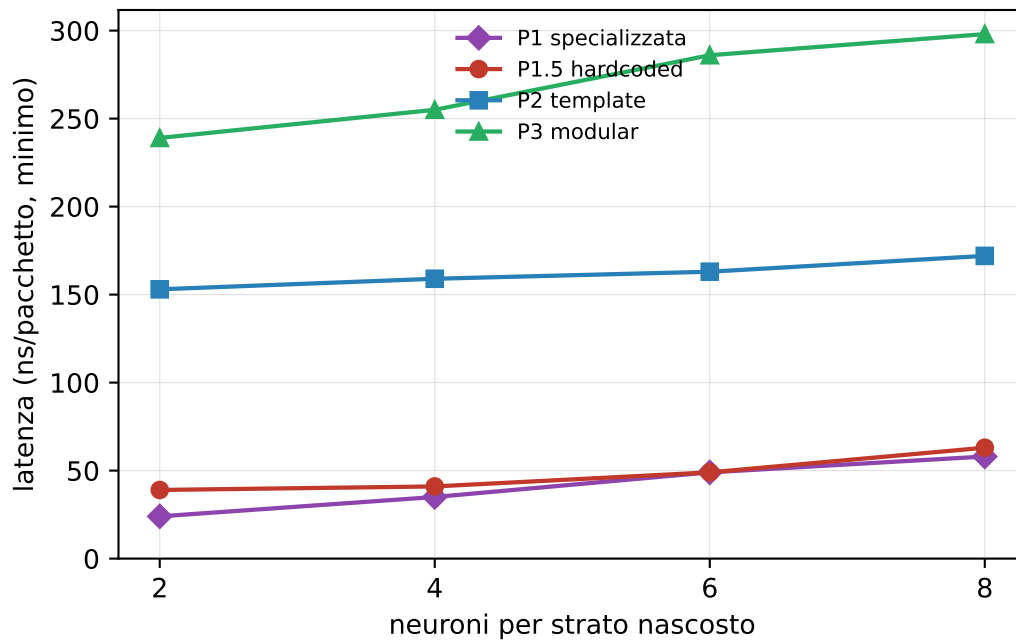
La taglia della *rete* entra nel programma solo in P1, che passa da 727 a 1 695 istruzioni fra 10 e 100 nodi: è lo switch della one-hot che si srotola. P2 e P3 restano alla cifra esatta a ogni punto, perché il loro codice non nomina mai la forma del modello.

E a runtime?



Piatte tutte e tre. Cento nodi costano quanto dieci. Il motivo è strutturale: una one-hot ha un solo uno, quindi il datapath legge *una sola colonna di pesi* qualunque sia la larghezza del vettore.

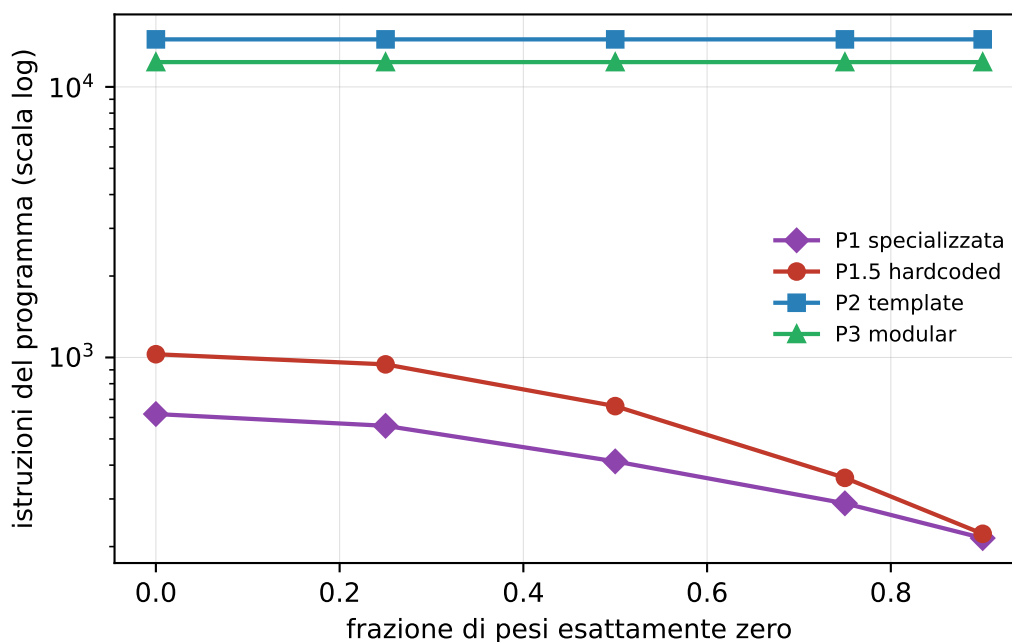
Allargare i layer *nascosti*, invece:



La rete può crescere quanto vuole; il modello no.

Allargare l'ingresso è gratis a runtime, perché una one-hot legge una colonna sola. Allargare i layer *nascosti* si paga, su tutte e tre le pipeline, perché ogni neurone in più sono pesi in più letti per ogni pacchetto.

11.9 I pesi contano, ma solo per una delle tre



Stessa identica forma in tutti e cinque i punti: cambia solo *quanti* pesi valgono zero.

Con il 90% dei pesi a zero, P1 passa da 1 028 a **223 istruzioni**. P2 e P3 non si muovono di una.

P1 scrive i pesi nel codice come numeri, quindi il compilatore vede una moltiplicazione per zero e la cancella. Per P2 e P3 uno zero è un byte in una tabella come un altro, e il programma è lo stesso.

Lo stesso vale per la composizione dell'ingresso: la figura del controllo, due sezioni più su, mostra che cambiare *quali* feature compongono il vettore d'ingresso ricompila entrambe le P1, mentre P2 e P3 leggono il descrittore da una tabella e non cambiano di una istruzione.

11.10 Il risultato che non cercavamo

Durante lo sweep una configurazione si è fatta rifiutare dal verificatore: quattro hidden layer da quattro neuroni. Le forme intorno caricavano tutte, inclusa quella *più grande* con cinque strati. Nessuna proprietà della profondità può produrre una cosa del genere.

Tenendo ferma la forma e cambiando solo il seme casuale dei pesi:

seme	esito
42	rifiutato
1	carica, 1 119 istruzioni
2	carica, 1 175
7	carica, 1 118
999	carica, 1 075

Il seme 1 carica a 1 119 istruzioni; il 42 viene rifiutato a 1 120. Una istruzione di differenza.

In P1, riaddestrare un modello senza toccare l'architettura può renderlo non caricabile. Stessa rete, stessi iperparametri, pesi nuovi, e il nodo lo rifiuta. P2 e P3 non hanno questo rischio: per loro i pesi sono byte in una tabella, e che il programma si carichi dipende solo dalla forma.

È il rovescio esatto del vantaggio di due pagine fa. La stessa riduzione del compilatore che porta P1 a 223 istruzioni è quella che lo rende imprevedibile. Un compromesso vero, misurato su entrambi i lati.

Il meccanismo preciso — *quale* limite del kernel — non è stato confermato, e non va inventato: il messaggio d'errore parla di un programma troppo lungo e cita un limite di 4096 istruzioni che nella stessa esecuzione P2 supera di quattro volte. È la stessa costante morta già incontrata sul capitolo del verificatore.

11.11 Che cosa questi grafici non dicono

- **Non si confrontano latenze prese in momenti diversi.** Nella misura precedente P2, a programma identico, dava circa 250 ns su due assi e circa 310 sugli altri due: la macchina si era caricata a metà esecuzione. Nella rimisura del 23 settembre sta fra 150 e 175 ns su tutti gli assi, e tutte le pipeline sono circa il 40% più veloci della misura precedente. Dentro una sessione i confronti reggono; fra sessioni reggono solo i rapporti. Le istruzioni, che sono deterministiche, si confrontano ovunque.
- La figura sulla larghezza va letta per la *direzione*, non per la pendenza, per lo stesso motivo.
- Oltre circa 115 nodi non si va senza alzare un tetto compilato e **rimisurare**.

Il costo del programma, misurato da solo

Prima di mettere un programma in una rete conviene sapere quanto costa da solo. Qui lo strumento è BPF_PROG_TEST_RUN: il kernel esegue il programma su un pacchetto preparato, in un ciclo, senza schede di rete, code o altri programmi di mezzo. È come cronometrare un cassiere mentre batte sempre lo stesso scontrino: misura il gesto, non la fila. La fila è l'argomento del capitolo 13.

Le sue misure non sono toccate dai difetti trovati sul test reale, ma sono state **rifatte il 23 settembre** con P1 come oggetto AOT, quello che va sui nodi. Tabelle, coefficienti e grafici qui sotto vengono da quella rimisura; le tabelle sulla correzione del banco, in fondo, restano quelle storiche perché documentano il prima e il dopo.

12.1 La campagna sulle architetture

I numeri fin qui riguardano *un* modello, il 65-4-4-7 depositato. Ma la domanda che interessa a chi deve scegliere un'architettura è un'altra: *come cresce il costo quando il modello cresce?*

Per rispondere servono più modelli misurati con la stessa metodologia, ed è la campagna: quattro assi, un solo parametro che si muove per asse, tutti dentro BPF_PROG_TEST_RUN con il minimo su sette prove.

Asse	Valori	Che cosa muove
iv_dense	n_{in} 5, 9, 13, 17	colonne d'ingresso tutte <i>dense</i> : ognuna è letta da una mappa e moltiplicata
iv_onehot	n_{in} 16, 32, 65	colonne d'ingresso in una <i>one-hot</i>
width_camp	65- $v-v-7$, v 4, 8, 16, 32	neuroni per strato nascosto
depth_camp	65-8...8-7, 1-4 strati	profondità, a larghezza fissa 8

Due muri, entrambi dichiarati

Due punti della campagna non si possono misurare, e il banco lo dice invece di inventare un numero.

Larghezza 16 e 32 su Pipeline 2 e 3. I soffitti sono compilati: T2_MAX_H1, T2_MAX_H2, ML1_MAX_H1 e MLH_MAX_H valgono tutti 8. E Pipeline 3 non rifiuta il caricamento: risponde XDP_PASS a runtime. Una cella a larghezza 16 *si caricherebbe, girerebbe, e restituirebbe una latenza* — di un programma che non calcola niente. È il modo peggiore di rompersi, perché il numero è plausibile. Il banco controlla i soffitti *prima* di compilare e segna la cella RIFIUTATO senza misurarla.

Larghezza 32 su Pipeline 1. Qui il soffitto non c'è, ma il programma srotolato è troppo grande. Con BCC il backend LLVM abortiva il *processo*; con l'oggetto AOT la compilazione con clang fallisce con degli errori (20). Ogni cella gira nel proprio sottoprocesso apposta: la cella esce CRASH e lo sweep continua.

Larghezza 16 si misura su hardcoded (141 ns), ed è un punto che Pipeline 2 e 3 non possono avere. Su p1_static invece, che con BCC si misurava (199 ns), con l'oggetto AOT

la compilazione **fallisce** (quattro errori di clang). La causa non è ancora stata letta: il banco conserva solo l'ultima riga dell'errore. Da verificare rigenerando quel sorgente sulla VM.

12.2 Il problema: le MAC nominali non predicono la latenza

Il modo naturale di mettere in relazione complessità e tempo è contare le moltiplicazioni-accumulo dalla forma del modello: $n_{in} \times h_1$ per il primo strato, e così via. Fatto così, non funziona.

Retta ai minimi quadrati sui quattro assi insieme, con le MAC nominali sull'asse delle ascisse:

Pipeline	ns per MAC nominale	r^2
p1_static	0,017	0,11
hardcoded	0,061	0,70
template	0,066	0,19
modular	0,086	0,11

Per Pipeline 3 il modello spiega l'11% della varianza: i punti sono una nuvola.

Il sintomo più chiaro sta sull'asse `iv_onehot`, dove il vettore d'ingresso quadruplica:

n_{in}	pesi	p1_static	hardcoded	template	modular
16	271	59 ns	58 ns	152 ns	292 ns
32	399	58 ns	58 ns	151 ns	296 ns
65	663	59 ns	59 ns	150 ns	284 ns

n_{in} per quattro, pesi per 2,4, e la latenza **non si muove su nessuna delle quattro**: P1 specializzata 59/58/59 ns, P1.5 58/58/59, P2 152/151/150, P3 292/296/284. Le istruzioni di Pipeline 1 intanto salgono da 1 291 a 2 015: la taglia statica cresce, il percorso eseguito no.

(Nella misura precedente due righe si alzavano al punto più largo, `hardcoded` del 22% e `template` del 28%. Nella rimisura sono piatte anche loro: era rumore.)

12.3 La soluzione: contare le MAC che vengono eseguite

La causa è nel datapath, ed è una riga sola. Il ramo che tratta una feature one-hot è questo:

```

} else if (code == FEAT_INGRESS_IF) {
    if (_raw_iface >= 1 && _raw_iface <= sz) {
        __u32 c = coff + (_raw_iface - 1);
        for (int j = 0; j < ML1_MAX_H1; j++)
            out[j] += LW_W(LW, woff + j * n_in + c);
    }
}

```

Una one-hot occupa `sz` colonne nella matrice dei pesi, ma nel datapath ne attiva *una*: il costo è h_1 addizioni, e non dipende da `sz`. Contarla come $sz \times h_1$ moltiplicazioni sovrastima, e sovrastima di quanto dipende da quanta one-hot c'è nell'ingresso — cioè cambia da asse ad asse. Ecco perché i quattro assi non cadevano sulla stessa retta.

Il banco calcola quindi due colonne: `macs`, nominali, e `macs_eff`, che conta le one-hot per quello che eseguono. Con la seconda sull'asse delle ascisse:

Pipeline	MAC eseguite		MAC nominali	
	ns/MAC	r^2	ns/MAC	r^2
p1_static	0,174	0,87	0,017	0,11
hardcoded	0,231	0,95	0,061	0,70
template	0,451	0,69	0,066	0,19
modular	0,909	0,92	0,086	0,11

Per Pipeline 3 si passa da $r^2 = 0,11$ a $r^2 = 0,92$, e p1_static da 0,11 a 0,87. Quattro assi costruiti in modi diversi collasano sulla stessa retta, una per pipeline.

Il costo per MAC eseguita è una proprietà misurabile della pipeline: circa 0,17 ns quando i pesi sono compilati come letterali, circa 0,91 ns quando vengono letti da una tabella. Un fattore $5,2 \times$ ($4,5 \times$ nella misura precedente, che era più lenta del 40% su tutte), ed è il prezzo della genericità espresso in una costante invece che in un aneddoto.

Il pavimento della baseline, che di MAC ne esegue zero, sta fra 16 e 24 ns: è il costo del framework XDP che ogni pipeline paga prima di moltiplicare qualunque cosa.

Un residuo che ha una spiegazione

Il residuo medio di Pipeline 3 è più alto sugli assi della larghezza (24 ns) e della profondità (14 ns) che sugli altri. È atteso: ogni strato in più è anche una *tail call*, e una tail call non è una MAC. Sull'asse delle istruzioni, invece, Pipeline 3 non ha nemmeno una pendenza da stimare — il suo conteggio è 12 349 su ogni cella, perché non ricompila mai, mentre la latenza varia da 238 a 455 ns. Il suo costo non sta nella taglia del codice.

12.4 Un difetto del banco, trovato dalla calibrazione

Resta da chiudere il cerchio. I due banchi a BPF_PROG_TEST_RUN misurano lo stesso programma: devono dare lo stesso numero. Per settimane non l'hanno fatto, e lo scarto non era nemmeno costante:

Pipeline	test_suite	campagna	scarto
baseline	23 ns	9 ns	−61%
hardcoded	49 ns	37 ns	−24%
template	177 ns	149 ns	−16%
modular	278 ns	240 ns	−14%

(misure del 21 settembre, prima della correzione descritta qui sotto.)

La causa: un pacchetto riusato centomila volte

Il banco parametrico misurava riusando **un solo pacchetto** per tutte le ripetizioni. E il kernel, dentro BPF_PROG_TEST_RUN, non ripristina il buffer fra una ripetizione e l'altra. Il datapath decrementa il TTL, che dopo una quarantina di esecuzioni restava inchiodato a 1: da lì in poi ogni ripetizione prendeva il ramo

```
if (ip->ttl <= 1) { /* ... */ return XDP_PASS; }
```

Quel ramo sta **dopo** l’inferenza. Il modello girava comunque — ed è esattamente il motivo per cui il difetto non si vedeva: le curve scalavano in modo regolare con la larghezza e con la profondità, come ci si aspetta da una misura sana. Ma saltava la coda di inoltro: decremento del TTL, checksum, pkt_stats, cls_stats, mac_table, bpf_redirect.

Il difetto riguardava **tutti e undici gli assi**, non solo la campagna: la funzione che campiona è una sola e la chiamano tutti e quattro i misuratori. Ogni latenza prodotta dal banco parametrico prima del 22 settembre misurava «parse più inferenza», non «tutto il percorso».

La correzione, e la prova che era quella giusta

Si usa prog_test_run_bench, la stessa funzione di test_suite: rinfresca il frame ogni 200 esecuzioni partendo da TTL 255. Il conto che chiude il buco è $255 - 200 = 55$, quindi nessuna ripetizione arriva alla scadenza.

Rimisurato tutto, nella stessa sessione:

Pipeline	test_suite	campagna	scarto
baseline	28 ns	30 ns	+7%
hardcoded	71 ns	79 ns	+11%
template	276 ns	271 ns	-2%
modular	453 ns	428 ns	-6%

Convergono, entro l’11% e senza segno sistematico. Lo scarto che resta su hardcoded non è rumore e ha una causa nota: test_suite usa i pesi veri del modello addestrato (1026 istruzioni), il banco parametrico pesi sintetici a sparsità nulla (1071). Sono due programmi diversi, quindi 71 contro 79 è ciò che ci si attende.

L’effetto collaterale, più importante del claim

La stessa correzione ha reso **confrontabili sweep diversi**. Il modello 65-4-4-7 compare sia come punto a 6 porte dell’asse degree sia come primo punto della campagna, con istruzioni identiche al bit — è lo stesso identico programma. Prima i due sweep divergevano di un fattore uniforme 1,8; adesso concordano entro il 2% su tre righe su quattro:

	asse degree	campagna	scarto
p1_static	58 ns	59 ns	+2%
hardcoded	69 ns	79 ns	+14%
template	268 ns	271 ns	+1%
modular	436 ns	428 ns	-2%

Era l’incoerenza più grave del banco, e si è chiusa insieme a questa.

Una validazione che non cercavamo

Le istruzioni sullo stesso modello, contate dai due banchi: baseline 155 e 155, template 14985 e 14985, modular 12349 e 12349. Identiche. L'unica che diverge è hardcoded, 1026 contro 1071, e diverge nella direzione giusta: i pesi veri hanno qualche zero, clang cancella quelle moltiplicazioni, 45 istruzioni in meno. È lo stesso effetto che l'asse sparsity misura apposta. E che i conteggi non si siano mossi di un'unità prima e dopo la correzione è la prova che questa ha toccato solo il cronometro, non i programmi.

Il test reale: quanto costa un pacchetto al nodo

Questo capitolo è stato riscritto da capo il 23 settembre. La versione precedente conteneva conclusioni che le misure successive hanno smentito, e correggerle una per una avrebbe reso il testo illeggibile. In fondo al capitolo c'è una tabella che mette le cifre vecchie accanto a quelle nuove.

13.1 La domanda, in parole semplici

Un nodo della rete riceve pacchetti, per ognuno esegue la rete neurale, decide da quale porta farlo uscire, e lo spedisce. La domanda è: **quanto tempo spende il nodo su ogni pacchetto**, dal momento in cui lo prende al momento in cui lo ha consegnato all'uscita?

Un'immagine per tutto il capitolo: la cassa del supermercato

Il nodo è un **cassiere**. I pacchetti sono i **clienti**. Il generatore di traffico è la porta da cui entrano i clienti, e il collegamento virtuale fra generatore e nodo è il **nastro** su cui i clienti aspettano: ha 256 posti, e quando è pieno chi arriva viene mandato via.

La domanda del capitolo è *quanti secondi spende il cassiere per ogni cliente*. Il modo più semplice di saperlo è far arrivare più clienti di quanti il cassiere riesca a servire, così che il nastro non sia mai vuoto, e contare quanti ne serve in un secondo. Se ne serve 5 milioni al secondo, spende $1/5\,000\,000$ di secondo a cliente, cioè 200 nanosecondi.

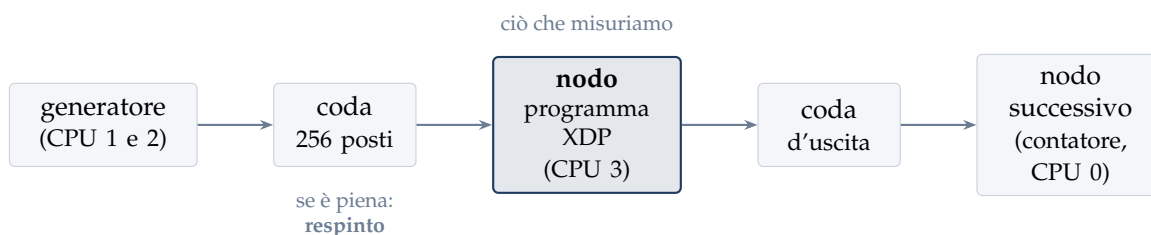
Il numero che cerchiamo è quindi il **costo per pacchetto in nanosecondi** (ns; un nanosecondo è un milionesimo di secondo). Si ottiene così:

$$\text{costo per pacchetto} = \frac{1 \text{ secondo}}{\text{pacchetti elaborati in un secondo}}.$$

Quando il nodo lavora senza sosta, e sul banco lo si fa apposta, questo numero è esattamente il tempo che il nodo spende su ciascun pacchetto.

13.2 Com'è fatto il banco

Tutto sta su una sola macchina virtuale con quattro processori (CPU). Per non misurare una cosa per un'altra, ogni parte ha il suo processore:



- **Il generatore** crea pacchetti uguali a quelli veri: stesso formato, stesso header IPA, stesso modello da eseguire.
- **La coda d'ingresso** è quella di un *veth*, il cavo virtuale di Linux. Ha 256 posti e la sua dimensione non si può cambiare.
- **Il nodo** è una delle pipeline (baseline, P1, P1.5, P2, P3) caricata in XDP sul processore 3. È l'unica cosa che si vuole misurare.
- **Il nodo successivo** è un programma che conta i pacchetti arrivati e li butta. Rappresenta chi riceve il pacchetto dopo di noi.

Oltre alle cinque pipeline il banco misura, nella stessa sessione, un sesto programma di riferimento: *rxonly*, che prende il pacchetto dalla coda e lo butta senza guardarlo. Non fa niente, quindi dice quanto costa *soltanto ricevere*: è il pavimento sotto il quale nessuna pipeline può scendere.

13.3 Le cinque trappole

Arrivare a un numero affidabile ha richiesto di trovare e togliere cinque difetti di misura. Nessuno riguardava le pipeline: tutti facevano sembrare il nodo diverso da com'è. Vengono raccontati nell'ordine in cui sono stati scoperti, perché ognuno era nascosto dal precedente.

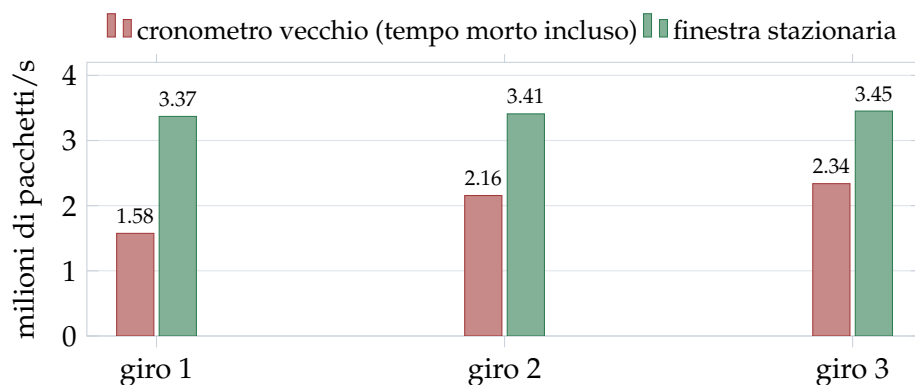
Trappola 1 — Il cronometro partiva prima dei clienti

Il generatore del kernel, *pktgen*, si avvia con un comando che *blocca* finché la trasmissione non è finita. Il banco misurava la durata di quel blocco e ci divideva i pacchetti. Ma il blocco non dura quanto la trasmissione: *pktgen* aspetta sempre 125 millisecondi fissi prima di partire, i suoi thread si accorgono del via con fino a 100 ms di ritardo, e alla fine controlla ogni 100 ms se hanno finito. In tutto, fino a circa **225 ms di tempo morto** su finestre di misura da 300 ms.

È come cronometrare un cassiere facendo partire il cronometro quando si apre il negozio invece che quando entra il primo cliente: il cassiere sembra più lento di quello che è, e di una quantità che cambia ogni volta.

La firma nei dati era evidente, una volta saputo dove guardare: nello stesso run, tre giri diversi davano 1 240 424, 1 238 514 e 1 240 533 pacchetti al secondo. Tre numeri quasi identici, perché erano sempre lo stesso conteggio diviso lo stesso tempo morto, arrotondato a passi da 100 ms.

Il rimedio è la *finestra stazionaria*. Si aspetta che tutti i thread del generatore stiano trasmettendo, si lascia assestare la coda, si leggono i contatori, si aspettano 300 ms, si rileggono, e solo allora si ferma il generatore. Il rate è la differenza fra le due letture divisa per il tempo fra le due letture. L'avvio e la chiusura restano fuori per costruzione.



Stessa pipeline (P1.5), stessi parametri, due run uno dopo l'altro. Con il cronometro vecchio i giri variano del 48%, con la finestra stazionaria del 2%.

Trappola 2 — Il cronometro che si fermava a metà lettura

Anche con la finestra stazionaria, al primo run vero una finestra ha dato 5,33 milioni di pacchetti al secondo mentre tutte le altre ne davano 3,4. La causa sta nella macchina: l'host può sospendere la macchina virtuale per qualche decina di millisecondi quando vuole. Se lo fa proprio mentre il banco legge i contatori, i contatori vanno avanti e l'orologio della misura no.

Il rimedio: ogni lettura (generatore più nodo) deve durare meno di 3 ms; se dura di più viene rifatta, e le si attribuisce l'istante a metà. Un test senza kernel, con un generatore simulato che riproduce i tempi di `pktgen`, lo verifica: simulando una sospensione di 150 ms dentro una lettura, il rate esce giusto entro il 2%. Togliendo la rilettura, esce sbagliato del 20%.

Trappola 3 — I clienti mandati via non erano contati

Quando la coda d'ingresso è piena, il collegamento virtuale rifiuta il pacchetto («respinto»). Una parte del banco contava solo i pacchetti *entrati*, e così scriveva «perdita 0,00%» anche quando metà del traffico veniva respinto.

Ma un pacchetto respinto è un pacchetto che il nodo non ha preso, perché non svuotava la coda abbastanza in fretta: **è una perdita del nodo**, non del generatore. Ora il banco li conta sempre e li mostra separati:

- **respinti** — mai entrati, la coda era piena;
- **persi dopo l'ingresso** — entrati ma non usciti.

Questa seconda colonna è rimasta fra lo 0,00 e lo 0,07% in tutti i run: di ciò che le pipeline prendono, non perdono praticamente niente.

Trappola 4 — Il generatore faceva lavorare il nodo

`pktgen` crea i pacchetti come li crea lo stack di rete di Linux: in una struttura chiamata *skb*. Il programma XDP però vuole il pacchetto «grezzo», con 256 byte liberi davanti; `pktgen` ne lascia 64. Per ogni pacchetto, quindi, il collegamento virtuale deve **copiarlo** in un'altra zona di memoria prima di darlo al nodo. E quella copia la paga il processore del nodo.

Su una scheda di rete vera con XDP nativo quella copia non esiste: il driver consegna il pacchetto già grezzo. Stavamo misurando un costo del banco come se fosse del nodo.

Il **rimedio** è un secondo generatore, scritto apposta (`ipa/test/xdp_gen.py`). Usa una funzione del kernel (`BPF_PROG_TEST_RUN` in modalità *live frames*) che crea pacchetti già grezzi e li spinge nel collegamento: niente `skb`, niente copia. I pacchetti sono identici a quelli di `pktgen`, byte per byte nei campi che le pipeline leggono.

Il risultato sul solo `rxonly`, cioè sul costo di ricevere:

	pacchetti al secondo	costo per pacchetto
con <code>pktgen</code> (<code>skb</code> + copia)	4,6–4,8 milioni	circa 210 ns
con frame grezzi	17,5–20,2 milioni	circa 50 ns

Circa tre quarti del costo di ricezione misurato con `pktgen` erano la copia.

Un difetto del generatore nuovo, trovato e tolto

Al primo run il generatore nuovo dava numeri molto dispersi (fino a $\pm 88\%$) e un fatto strano: più il generatore riusciva a offrire, più il nodo sembrava veloce, come se fossero legati. La causa: il kernel ricicla le pagine di memoria dei pacchetti da una riserva che, con le impostazioni di default, ne tiene solo 64 e viene ricreata ogni 65536 pacchetti. Le pipeline tengono in circolo molti più pacchetti di così, e la riserva si svuotava. Portando la riserva al massimo consentito (256) e le chiamate a un milione di pacchetti, la dispersione è scesa a pochi punti percentuali.

C'era anche un errore mio: i primi pacchetti erano lunghi 60 byte invece di 64, perché avevo tolto 4 byte di troppo. L'header IPA ne vuole 21 e ne restavano 18: le pipeline scartavano il pacchetto come malformato. Se ne è accorta la *sonda* che il banco esegue prima di misurare, e che esclude una pipeline che non riconosce il pacchetto invece di misurarle un lavoro che non fa.

Trappola 5 — Il nodo successivo lavorava sul processore del nodo

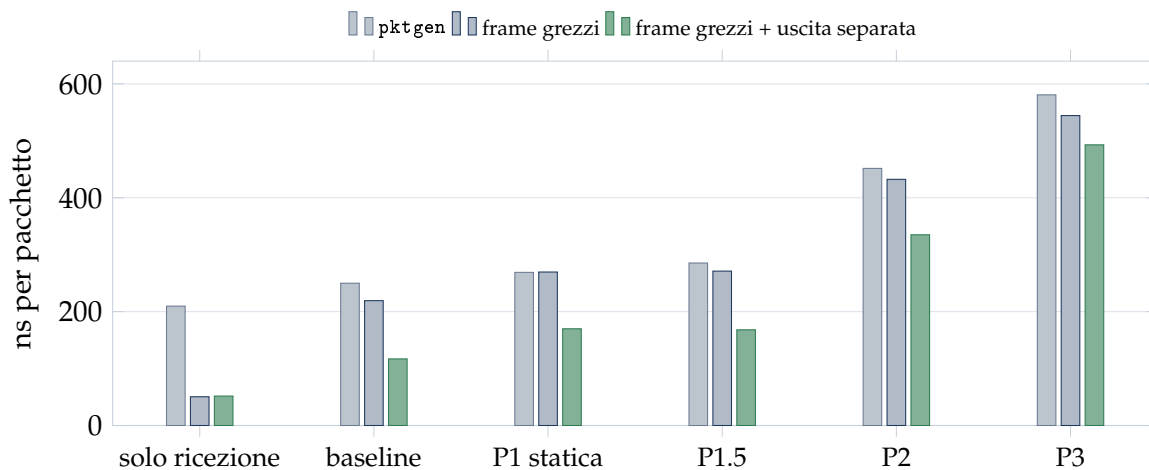
Nel banco il nodo successivo, cioè il contatore d'uscita, girava sullo stesso processore del nodo. Ogni pacchetto inoltrato costava quindi al processore 3 anche la *ricezione dall'altra parte*: come se il cassiere, dopo ogni cliente, dovesse anche imbustargli la spesa.

Il **rimedio**: il contatore d'uscita gira ora su un altro processore (opzione `--egress-cpu 0`). Sul processore del nodo resta solo il lavoro del nodo: prendere il pacchetto dalla coda, eseguire la rete, decidere, accodarlo sull'uscita.

Il nodo successivo, su un processore che fa anche altro, a volte non tiene il passo e perde qualche pacchetto (dallo 0,2 al 4,5%). Il nodo quei pacchetti li ha comunque elaborati, quindi il costo si calcola dai **pacchetti elaborati** dal programma (qualunque sia la decisione), non da quelli arrivati dall'altra parte.

Perché con `pktgen` questo non funziona

Separando l'uscita con `pktgen` il nodo è diventato *più lento* (da 4,0 a 2,2 milioni al secondo), invece che più veloce. L'ipotesi, non verificabile dal repository: con `pktgen` il nodo successivo resta spesso senza lavoro e si addormenta, e ogni volta il nodo deve svegliarlo da un altro processore, cosa che su una macchina virtuale costa molto. Con i frame grezzi il nodo successivo non si addormenta mai. Per questo la misura del costo del nodo si fa **solo** con il generatore nuovo.



Costo per pacchetto sul processore del nodo, 64 byte, a saturazione. Da sinistra a destra in ogni gruppo si tolgono le due contaminazioni: la copia del generatore e il nodo successivo. La terza barra è il costo del nodo.

13.4 Il risultato: quanto costa un pacchetto

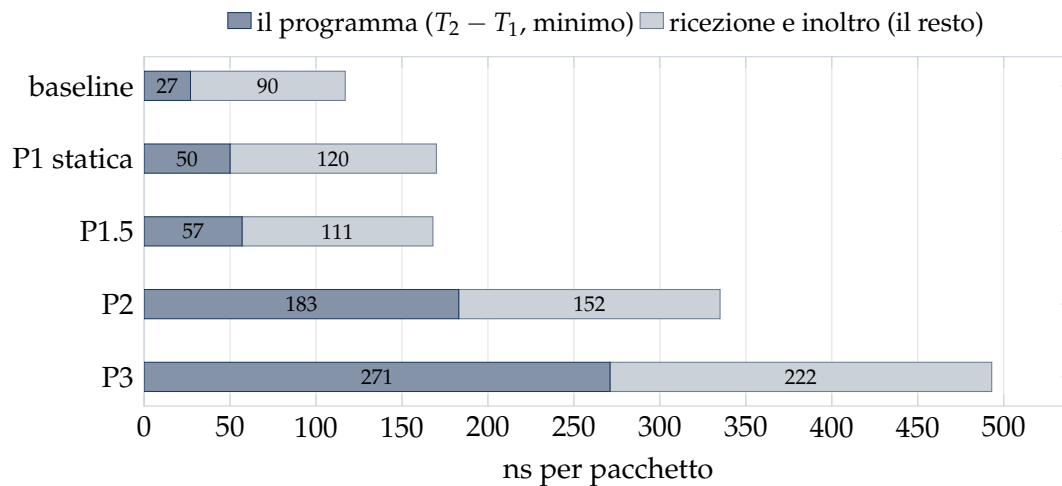
Con le cinque trappole tolte, il costo per pacchetto sul processore del nodo, dalla presa del pacchetto alla consegna sull'uscita, è questo (5 giri, pacchetti da 64 byte, controllo di validità superato):

Pipeline	costo per pacchetto	sopra la baseline	pacchetti/s per CPU
baseline (nessuna rete)	117 ns	—	8,5 milioni
P1 statica	170 ns	+53 ns	5,9 milioni
P1.5 (hardcoded)	168 ns	+51 ns	6,0 milioni
P2 (template)	335 ns	+218 ns	3,0 milioni
P3 (modular)	493 ns	+376 ns	2,0 milioni

La **baseline** riceve, legge l'intestazione, decrementa il TTL e inoltra: 117 ns. Tutto ciò che le pipeline aggiungono sopra è il **costo della rete neurale**: circa 50 ns quando i pesi sono scritti nel programma (P1, P1.5), circa 220 ns quando la forma della rete è letta da una tabella (P2), circa 375 ns quando anche la profondità lo è (P3). Una rete configurabile a runtime costa quindi fra 3 e 4 volte la baseline; una compilata, 1,4 volte.

Il programma, e tutto il resto

Il capitolo precedente misurava il solo programma, con tre marcature di tempo dentro il datapath ($T_2 - T_1$: dall'ingresso del programma all'istante prima di inoltrare). Mettendo le due misure una accanto all'altra si vede quanto del costo del nodo è il programma e quanto è ricezione e inoltra:



La parte «resto» cresce con la complessità della pipeline (da 90 a 222 ns). Non è un errore: $T_2 - T_1$ è il *minimo* osservato, il caso più fortunato, mentre il costo del nodo è la *media* sotto carico continuo, dove letture di tabella e salti fra programmi costano di più perché la memoria veloce del processore è contesa. **Per dimensionare un nodo vale il costo del nodo;** per capire dove va il tempo dentro il programma vale $T_2 - T_1$.

13.5 Quanto è solido questo numero

Un numero misurato una volta sola su una macchina virtuale non vale molto. Lo stesso costo è stato misurato cambiando, una alla volta, le condizioni che potevano spostarlo.

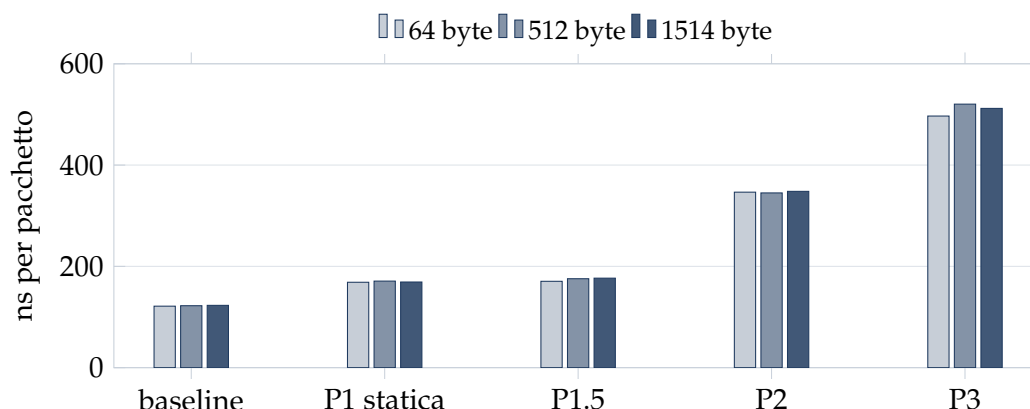
Da una sessione all'altra

Pipeline	sessione 1	sessione 2	sessione 3	sessione 4
baseline	117	121	121	122
P1 statica	170	164	169	165
P1.5	168	169	171	169
P2	335	341	347	312
P3	493	493	497	466

ns per pacchetto, 64 byte. Quattro sessioni dello stesso giorno con condizioni diverse (pacchetto fisso, TTL variabile, taglie diverse, classe 0).

Le differenze stanno entro circa il $\pm 5\%$. E la parte che conta di più, cioè quanto le pipeline costano *sopra la baseline*, si ripete anche con il generatore vecchio: P2 +202, +213, +215, +218 ns in quattro run; P3 +331, +325, +364, +376 ns.

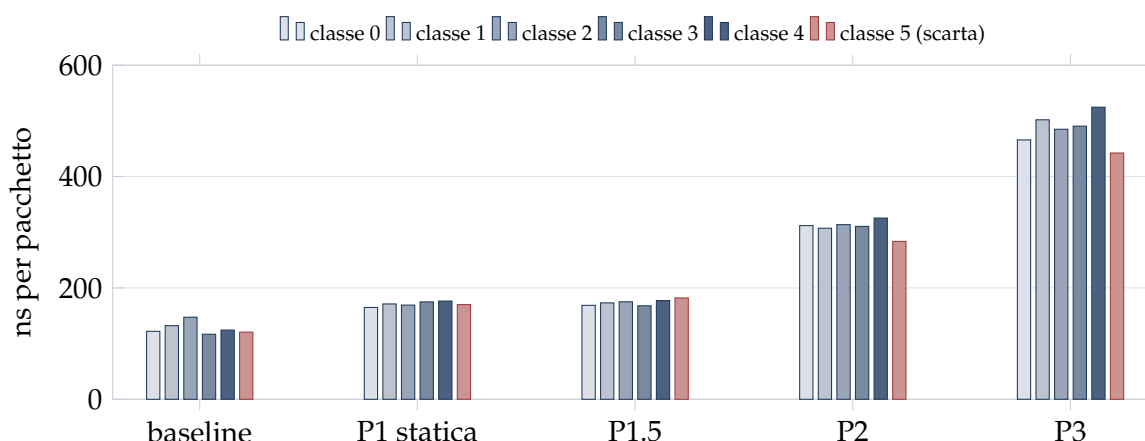
Con pacchetti di taglia diversa



Il costo non cambia con la taglia del pacchetto. È quello che succede su una scheda di rete vera con XDP nativo: il nodo guarda solo l'intestazione, che è lunga uguale in un pacchetto piccolo e in uno grande. Quindi il costo è *per pacchetto*, e il traffico in bit al secondo cresce con la taglia.

Con decisioni diverse

Un pacchetto sempre uguale fa decidere alla rete sempre la stessa cosa: una sola porta d'uscita, sempre la stessa voce di tabella. Per vedere se il costo cambia con la decisione, il banco (opzione `--per-class`) cerca per ogni classe che il modello sa produrre uno stato dei collegamenti che la faccia scegliere, lo imposta su tutte le pipeline e misura. Alla fine controlla che ogni pipeline abbia davvero deciso la classe voluta: lo hanno fatto tutte, in tutte e sei le classi.



- **La porta d'uscita non conta:** le cinque classi che inoltrano costano uguale entro circa il 10%, che è il rumore fra sessioni (la baseline, che fa sempre la stessa identica cosa, oscilla fra 117 e 147 ns).
- **Scartare non costa sensibilmente meno che inoltrare.** Il pacchetto scartato non viene accodato sull'uscita, ma il processore del nodo deve restituire la sua pagina di memoria al generatore, cosa che per un pacchetto inoltrato fa il nodo successivo. È un effetto del banco, non della pipeline.

- **Il TTL non cambia niente:** variandolo da 2 a 33 a ogni pacchetto, il costo resta lo stesso e la rete decide sempre la stessa classe. Questo modello decide in base al nodo e allo stato dei collegamenti, non al TTL (lo si sapeva già dal test di consegna). Che il generatore scriva davvero il TTL è provato: con TTL 1 i pacchetti scaduti compaiono fra gli scartati.

Il costo del nodo dipende dalla **pipeline** e da niente altro di ciò che è stato variato: né dalla sessione, né dalla taglia del pacchetto, né dalla classe decisa, né dalla porta d'uscita, né dal TTL.

13.6 Quello che su questa macchina non si può risolvere

I respinti anche quando il nodo non è pieno

Offrendo a tutte le pipeline 1,55 milioni di pacchetti al secondo, cioè meno di quanti ciascuna ne regga, una piccola parte viene comunque respinta: dall'1 al 3,5% per le più leggere, fino al 10% per P3. Anche `rxonly`, che non fa niente, ne respinge l'1-2%.

La causa è il risveglio. Quando la coda si svuota il processore del nodo si addormenta; quando arriva un pacchetto va svegliato, e su una macchina virtuale il risveglio passa dall'host e può richiedere centinaia di microsecondi. Nel frattempo 256 posti si riempiono in 170 microsecondi.

Tre rimedi provati, tutti e tre esclusi:

Tentativo	Esito
allargare la coda	il kernel della macchina non lo permette (<code>ethtool -G</code> non supportato sul veth)
tenere il thread del nodo sempre sveglio (<i>busy poll</i>)	non disponibile su questo kernel (il valore 2 di <code>threaded</code> viene rifiutato anche con il thread presente)
tenere sveglio il processore con un ciclo a priorità minima	peggiora tutto: la capacità di <code>rxonly</code> scende da 4,6 a 2,7 milioni al secondo e i respinti salgono dall'1 al 35%; rallenta anche il generatore, cioè l'intera macchina

Resta un limite dichiarato del banco. Non tocca il costo del nodo, che si misura a coda sempre piena, e `rxonly` ne misura l'entità in ogni sessione: ciò che una pipeline respinge *oltre* quella soglia è suo.

13.7 Che cosa questo numero non dice

- **È una macchina virtuale.** Il processore è ibrido (core veloci e core lenti) e la macchina virtuale non sa su quale gira. I *rapporti* fra le pipeline sono solidi; le cifre assolute valgono per questa macchina.
- **«Ricevere» e «inoltrare» sono quelli di un cavo virtuale.** Prendere il pacchetto dalla coda del veth non è prenderlo dalla memoria di una scheda di rete, e accodarlo sul veth successivo non è scrivere un descrittore di trasmissione. Su hardware vero questi due estremi costano cifre diverse.
- **Un modello solo.** Tutte le misure di questo capitolo usano il modello depositato (65-4-4-7). Come il costo cresce con modelli più grandi lo dice la campagna del capitolo precedente, ma a livello di solo programma: su traffico vero non è stato misurato.

13.8 Le cifre vecchie e quelle nuove

Si diceva	Oggi	Perché è cambiato
le pipeline consegnano 1,1–1,4 milioni al secondo, «limite inferiore»	il nodo regge da 2,0 (P3) a 5,9 (P1) milioni al secondo per processore	cronometro con tempo morto (trappola 1) e copia del generatore (trappola 4)
tetto della via di ricezione 3,7 milioni al secondo	4,8 con pktgen, 19,8 con frame grezzi	stesso cronometro; e la copia era tre quarti del costo di ricezione
le pipeline hanno ancora «circa 2×» di margine	smentito : il nodo era già saturo	il tetto era misurato su un percorso senza inoltro, e con il cronometro sbagliato
perdita 0,000% su ogni riga	respinti dall'1 al 94% a seconda del carico; persi dopo l'ingresso 0,00–0,07%	i respinti sono perdita del nodo (trappola 3)
«il datapath non perde, perde il trasporto»	vera solo la metà: dopo l'ingresso non si perde niente	un respinto è un pacchetto che il nodo non ha preso
$T_2 - T_1 = 27 / 50 / 57 / 183 / 271$ ns	invariato	è una misura con l'orologio del kernel, non tocca nessuna trappola

Le altre correzioni di settembre

Il test reale non è stata l'unica cosa sistemata nell'ultima settimana. Questo capitolo racconta le altre, una sezione ciascuna, con lo stesso schema: che cosa non andava, perché contava, che cosa è stato fatto, e come si è verificato che fosse davvero a posto.

14.1 Il P1 che va sui nodi non inoltrava

Il contesto. P1 si può costruire in due modi. Con *BCC*, che compila il programma sul nodo al momento del caricamento (serve un compilatore sul nodo, e ci vuole più di mezzo secondo). Oppure *AOT* (*ahead of time*): si compila prima, su un'altra macchina, e al nodo si spedisce l'oggetto già pronto, che un piccolo caricatore in C apre e attacca. È la versione AOT quella che va davvero sui nodi.

Il difetto. Il caricatore AOT riempiva la tabella delle uscite (quale interfaccia corrisponde a ogni porta) con l'interfaccia numero 1 per tutte le porte. L'interfaccia 1, su Linux, è *loopback*: il collegamento interno della macchina con sé stessa. Ogni pacchetto che la rete decideva di inoltrare veniva mandato lì e perso. Nessun test se ne accorgeva, perché i test di consegna provavano la versione BCC, non quella che va sui nodi.

La correzione. Il caricatore ora lascia le tabelle vuote e le «appunta» nel file system del kernel; il pannello di controllo in Python le riempie con le interfacce vere del nodo e avvia il controllo dello stato dei collegamenti, poi dice al caricatore di attaccare il programma. Il caricatore stacca tutto e pulisce quando riceve lo stop.

La prova. Il test di consegna ha ora una variante che usa esattamente il percorso di deploy: **11 casi su 11**, con ogni classe consegnata sull'interfaccia giusta e lo scarto contato.

14.2 Tutti i test di P1 usano ora l'oggetto AOT

Una volta sistemato il deploy, restava un'incoerenza: i test e le misure di P1 giravano ancora sulla versione BCC, che sui nodi non va mai. Ora **ogni** test e ogni banco che tocca P1 o P1.5 usa l'oggetto AOT, attraverso un modulo solo (`ipa/p1_aot.py`): la prova di consegna, le prove sul kernel, i modelli sintetici, la semantica delle classi, l'analisi parametrica, i banchi su traffico vero.

Risultati sulla macchina virtuale, tutti verdi:

Prova	esito
consegna su rete di veth (tutte le pipeline)	29/29
semantica delle classi, dentro il kernel	72/72
modelli generati, dentro il kernel	62/62
equivalenza numerica sui modelli sintetici	8/8 scenari, 300/300 ingressi ciascuno
P1 statica contro P1.5: stessa decisione	80/80 casi

14.3 Il cronometro del caricatore AOT

Il difetto. La cifra «74 ns per l'AOT contro 73 ns per BCC», usata per dire che l'AOT non costa niente in più, era sbagliata. Il caricatore misurava eseguendo il programma un milione

di volte *sullo stesso pacchetto*, e il kernel non rimette a posto il pacchetto fra un'esecuzione e l'altra. Il programma decrementa il TTL: dopo poche decine di giri il TTL scadeva, e da lì in poi si misurava il ramo corto «pacchetto scaduto», non l'inoltro. È lo stesso difetto che il capitolo precedente racconta per il banco parametrico.

La correzione. Pacchetti nuovi ogni 200 esecuzioni, partendo da TTL 255, come nel resto dei banchi.

Il risultato rimisurato, stessa sessione:

	BCC	AOT
istruzioni	1 026	1 011
costo del programma	44 ns	46 ns
aggiornare il modello sul nodo	741 ms	9–10 ms

La conclusione di prima regge, ma adesso poggia su un numero giusto: l'oggetto AOT costa quanto il build BCC per pacchetto, e toglie dal nodo quasi tutto il costo dell'aggiornamento.

14.4 Due implementazioni che devono dare lo stesso numero

Il rischio. Il progetto ha un *riferimento*: il calcolo della rete scritto in Python, con gli stessi numeri interi del kernel, usato dai test per sapere quale decisione aspettarsi. Se il riferimento e il codice generato per il kernel sbagliano *allo stesso modo*, ogni test passa. Se sbagliano in modo diverso, ma ognuno è coerente con sé stesso, i test di ciascuno passano lo stesso.

Il difetto trovato. Il riferimento usato per i modelli sintetici non moltiplicava il bias per la scala giusta (lo schema intero è: il bias dello strato l va moltiplicato per s^l). Sul modello depositato non si vedeva.

La correzione. La scala è ora un argomento obbligatorio del riferimento, e ogni file di risposte attese dichiara lo schema e la scala con cui è stato calcolato: un file vecchio viene rifiutato invece di essere usato. In più c'è un controllo nuovo che confronta direttamente **il riferimento Python con il codice C generato per P1**, letto e valutato come testo, logit per logit, sul modello minimo e su tutti i modelli sintetici. Per provare che il controllo sa fallire, lo si è fatto girare su codice alterato apposta: si accorge dell'alterazione.

14.5 La scala delle feature in P2 e P3

Il TTL entra nella rete diviso per una costante scelta durante l'addestramento. Era già stato corretto che questa costante venga dal modello e non sia fissata a 30 nel codice. Mancava la prova *nel kernel* per P2 e P3. Ora c'è, con un controllo negativo: su un modello addestrato con scala 16, se si altera la scala scritta nella tabella del nodo le decisioni cambiano (41 casi su 300 nel modello `ipa_tt116`, 89 nel modello `sma11`); con la scala giusta tornano identiche al riferimento. Quindi la scala viene davvero letta e usata.

Due correzioni vicine, sullo stesso tema:

- P2 rifiutava un modello con un numero di classi d'uscita diverso da quello configurato; ora il numero viene dalla semantica dichiarata del modello.
- In P2 e P3 la tabella che traduce la classe in azione è una sola per tutti i modelli caricati. Ora il caricamento di un secondo modello con una semantica diversa viene rifiutato, invece di sovrascrivere in silenzio quella del primo.

14.6 Pulizia del repository

Tolto ciò che non corrispondeva più al codice:

- `ipa/test/diag_fabric_divergence.py`: diagnosi di un difetto del riferimento già chiuso; i casi scritti nel codice erano quelli del riferimento difettoso;
- la vecchia funzione di latenza del banco di throughput, che nessuno chiamava più.

Sono stati invece *tenuti*, dopo averli controllati, tre script che sembravano superati:

- `diag_p3_bisect.py`: serve a rifare una misura citata sui limiti del verificatore;
- `test_ipa.py`: è l'unico che invia traffico con più modelli verso un nodo vero;
- `bench_tailcall_overhead.py`: misura un costo che nessun altro isola.

Il README è stato riallineato ai conteggi reali dei test e ai file che esistono.

I numeri di oggi, in una pagina

	baseline	P1 statica	P1.5	P2	P3
<i>Il nodo su traffico vero (capitolo 13)</i>					
costo per pacchetto	117 ns	170 ns	168 ns	335 ns	493 ns
pacchetti al secondo per processore	8,5 M	5,9 M	6,0 M	3,0 M	2,0 M
costo della rete (sopra la baseline)	—	53 ns	51 ns	218 ns	376 ns
<i>Il solo programma</i>					
$T_2 - T_1$ sul percorso vero (minimo)	27 ns	50 ns	57 ns	183 ns	271 ns
BPF_PROG_TEST_RUN	17 ns	36 ns	42 ns	173 ns	258 ns
istruzioni del programma	155	618	1 028	14 985	12 349
<i>Cambiare modello su un nodo in servizio</i>					
caricare sul nodo	—	5–8 ms (AOT)		4–10 ms	
anche generare e compilare	—	70–140 ms		non serve	
serve un compilatore sul nodo?	—	no (compilato prima)		no	

In tre righe:

1. **Scrivere i pesi nel programma costa circa 50 ns per pacchetto; leggerli da una tabella circa 220–380 ns.** È il prezzo della configurabilità, e non dipende dal traffico.
2. **Con l'oggetto precompilato, cambiare modello costa lo stesso tempo per tutte le pipeline:** pochi millisecondi sul nodo, e meno di un decimo di secondo per P1 anche contando la compilazione. Il vecchio «1,5 secondi» era il costo di BCC, non del compilatore. Il vantaggio residuo di P2 e P3 è non dover compilare niente, da nessuna parte.
3. **Congelare anche il nodo (P1 statica) non porta un vantaggio misurabile** rispetto a P1.5: 170 contro 168 ns sul nodo, 80 decisioni su 80 identiche.

Da dove vengono i numeri della tabella

Le righe del nodo: `bench_throughput --mode compare --generator xdp --egress-cpu 0`, 23 settembre, cinque giri. $T_2 - T_1$: la campagna con tre marcature di tempo (`claims.md`, E3). BPF_PROG_TEST_RUN, istruzioni e aggiornamento: la campagna parametrica rifatta il 23 settembre con P1 come oggetto AOT (modello 65-4-4-7 a pesi sintetici).

Attenzione: le tre righe di latenza vengono da tre sessioni diverse, e questa macchina cambia velocità da una sessione all'altra (la rimisura parametrica è uscita circa il 40% più veloce della precedente su tutte le pipeline). Dentro una riga le cifre si confrontano; fra righe diverse si confrontano solo i rapporti.

Dopo il 24 settembre (semantica delle classi per modello, capitolo 3) P2 e P3 contano 14976 e 12374 istruzioni invece di 14985 e 12349, con le stesse letture di tabella e latenza entro il rumore; le loro tabelle occupano 128 KiB in più. Le altre righe non cambiano.

Quel che resta

16.1 Fatto

- semantica delle classi dichiarata, mai dedotta; catena classe → azione → porta logica → interfaccia;
- motore indipendente dallo scenario, dimostrato su cinque topologie diverse;
- emulatore rimosso; verifica end-to-end su datapath reale con XDP native;
- errore del bias nell'ultimo strato corretto;
- feature dell'interfaccia d'ingresso resa viva in tutte e tre le pipeline;
- **indice del nodo** letto a runtime da tabella in tutte e tre: era l'ultimo ingresso su cui le pipeline non concordavano, e pesa 208 pesi su 319;
- primo strato riscritto con i cicli invertiti in P2 e P3: stesso risultato bit per bit, un terzo dei rami in meno, e il margine di verifica che ne è uscito ha permesso di **recuperare il tetto sulle code** (da 1 a 8) e di **rimettere la riga delle letture di tabella**, che era n.d.;
- oggetto precompilato (AOT) compilato, caricato e misurato: aggiornamento del modello sul nodo da 741 ms a 9–10 ms, a parità di costo per pacchetto; e rimisurato su tutti gli assi, 5–8 ms, pari a P2 e P3;
- tabella dei risultati rifatta su uno stato del codice solo;
- **analisi parametrica** su cinque assi e tre pipeline: le pendenze sono diverse, ed è la misura che giustifica l'esistenza di tre pipeline invece di una;
- **versione P1 specializzata**: congela anche l'indice del nodo, e toglie alla pipeline hardcoded la dipendenza dalla taglia della rete — al prezzo di un binario per nodo;
- **semantica delle classi per modello** in P2 e P3: modelli con significati diversi per la stessa classe convivono nello stesso programma, con una sola lettura per pacchetto come prima e latenza invariata; costa 128 KiB di tabelle per pipeline (capitolo 3);
- P2 non è più fissa a due hidden layer: la profondità è diventata un involuppo compilato come già lo erano le larghezze — al prezzo, onesto e misurato, di una ricompilazione per profondità.
- **banco end-to-end** con generatore di traffico e nodo sotto prova su core separati (NAPI in modalità thread), condizioni rese identiche fra le pipeline — un fabric solo, tutto compilato prima di misurare, misura a giri — e un **controllo di validità** che il banco esegue su sé stesso: se la baseline non risulta la più veloce, il run viene dichiarato non valido invece che pubblicato;
- **tempo fra arrivo e partenza** misurato per tutte e cinque le pipeline, riproducibile entro pochi nanosecondi, e confermato da due sorgenti di traffico completamente diverse (202 contro 201 ns): è la misura che mancava, ed è quella che dice in nanosecondi quanto costa leggere il descrittore a runtime;

- **dichiarata l’inaffidabilità della portata** su questo banco: ogni riga porta la propria dispersione fra i giri, e sopra il 25% il numero viene marcato invece che riportato come se fosse un risultato;
- **il costo del nodo misurato**, dalla ricezione all’inoltro, dopo aver tolto cinque difetti di misura (capitolo 13): 117 / 170 / 168 / 335 / 493 ns, stabile fra sessioni, taglie di pacchetto e classi decise;
- **P1 AOT consegna davvero** sui nodi, e tutti i test di P1 usano ora l’oggetto che va in produzione (capitolo 14).

16.2 Aperto

- **I respinti sotto capacità.** Anche senza nessuna rete neurale, una piccola parte dei pacchetti (1–2%) viene respinta dalla coda del collegamento virtuale quando il nodo non è pieno: il risveglio del processore del nodo passa dall’host. Coda non allargabile, *busy poll* non disponibile, e tenere sveglio il processore peggiora la macchina. Resta un limite dichiarato, misurato in ogni sessione da *rxonly*.
- **Architetture diverse su traffico vero:** il costo del nodo è misurato su un modello solo. Come cresce con modelli più grandi è misurato solo a livello di programma (capitolo 12).
- **Scartare contro inoltrare:** sul banco scartare non costa meno che inoltrare, perché la restituzione della pagina di memoria ricade sul nodo. Su una scheda di rete vera il conto sarebbe diverso, e qui non si misura.
- **Versione intermedia fra P1 e P2:** esiste ed è stata misurata (*hardcoded*, pesi nel codice e nodo letto da mappa). Il risultato è negativo in senso utile: **non si distingue da P1**, né sul solo programma né sul costo del nodo (168 contro 170 ns).
- **Hardware vero:** la scheda di rete di questa macchina virtuale è emulata e non supporta XDP native. I numeri assoluti restano non citabili come prestazioni di sistema.
- **Il tetto sull’uscita di P2:** lo strato finale è srotolato a 32 classi per un modello che ne ha 7, cioè 256 moltiplicazioni dove ne servono 28. Abbassarlo ridurrebbe molto il programma, ma *cambia il comportamento*: restringe l’insieme dei modelli rappresentabili. La strada pulita è generare quel tetto dall’inviluppo dichiarato dal deployment invece di fissarlo nel sorgente — una ricompilazione per inviluppo, non per modello.
- **I salti fra programmi di P3:** sono tre per pacchetto ed è lì che sta la latenza, non nella dimensione. Ridurli è un lavoro diverso da quello fatto finora.
- **Perché quel modello non si carica:** sappiamo che dipende dai pesi e non dall’architettura, ma non *quale* limite del kernel venga toccato. Il messaggio d’errore ne nomina uno sbagliato. Serve il registro del verificatore, ed è un’ora di lavoro, non una settimana.
- **Rimisurare a macchina scarica:** metà delle latenze dell’analisi parametrica sono state prese mentre la VM era occupata. Le istruzioni no, quelle sono deterministiche.

16.3 Il filo conduttore

Guardando indietro, quasi tutti i problemi trovati a settembre hanno la stessa forma. Non erano calcoli sbagliati: erano **assunzioni non dichiarate** che risultavano vere per caso nell’unica configurazione mai provata.

Assunzione	Vera solo perché
classe k = interfaccia k	le porte erano numerate come le classi
DROP = ultima classe	nel modello depositato capitava così
$n_{\text{nodi}} = 52$ di default	si provava una topologia sola
eth0 = ifindex 2	si provava dentro un container pulito
bias dell'uscita $\times s^0$	si provava coi link tutti attivi
one-hot del nodo = model_id	si registrava un modello solo
tabella vuota = nodo 0	l'array è preallocato a zero
si carica $\Rightarrow$ dipende dall'architettura	non si era mai cambiato solo i pesi

E quasi tutte sono emerse allo stesso modo: cambiando *una* condizione che nessun test aveva mai cambiato.

Un test verde dice che il codice si comporta come il test si aspetta. Non dice che il test stia chiedendo qualcosa.

I comandi

Tutti i comandi si lanciano dalla radice del repository. Le convenzioni sono due: `sudo` serve ovunque si carichi un programma nel kernel o si tocchi un'interfaccia; senza `sudo` girano solo le prove che non escono da Python.

Prerequisiti

Linux con BCC installato. Per il banco di throughput anche il modulo del generatore:

```
sudo modprobe pktgen
```

Su macchina senza BCC le prove che toccano il kernel vengono saltate con un avviso, non falliscono.

17.1 Correttezza: il modello fa quello che deve

Non serve root: girano ovunque, anche su una macchina senza kernel Linux adatto.

```
# tutte le prove disponibili (quelle sul kernel vengono saltate senza BCC)
python3 ipa/test/test_suite.py

# una famiglia alla volta
python3 ipa/test/test_suite.py --only core --verbose
python3 ipa/test/test_suite.py --only quant --samples 200
python3 ipa/test/test_suite.py --only pktstats
python3 ipa/test/test_suite.py --only extract
python3 ipa/test/test_suite.py --only robust

# la catena classe -> azione -> porta logica -> interfaccia
python3 ipa/test/test_class_semantics.py

# topologie sintetiche: il motore non deve dipendere dallo scenario
python3 ipa/test/test_synth.py

# modelli sintetici, le sole vie Python (nessun kernel, nessun root):
# costruzione degli ingressi e confronto float / int8
python3 ipa/test/verify_synth_kernel.py --all --dry-run

# che cosa decide il modello, senza kernel di mezzo
python3 ipa/test/diag_model_decisions.py
```

Le famiglie disponibili per `-only` sono `core`, `pktstats`, `extract`, `quant`, `robust`, `kernel`, `all`.

17.2 Kernel: si carica, e quanto costa

Qui si misura il numero di istruzioni eBPF, la latenza per pacchetto e i salti fra programmi, con `BPF_PROG_TEST_RUN`. È il banco del *tapis roulant*: misura il programma, non la rete.

```
# la tabella delle metriche in-kernel per tutte le pipeline
sudo python3 ipa/test/test_suite.py --only kernel
```

```
# piu' ripetizioni se la macchina e' rumorosa
sudo python3 ipa/test/test_suite.py --only kernel \
    --kernel-repeat 200000 --kernel-trials 15

# piu' modelli caricati insieme: si smistano davvero per model_id?
sudo python3 ipa/test/verify_multi_model.py

# ogni modello la sua semantica: classe -> azione per (model_id, classe)
sudo python3 ipa/test/verify_per_model_semantics.py

# quanto costa, contro la versione a classe sola (albero di riferimento)
sudo python3 -u ipa/test/bench_semantics_models.py \
    --baseline-dir /tmp/ipa_base/ipa --trials 21

# le stesse prove di semantica e topologia, ma dentro al kernel
sudo python3 ipa/test/test_class_semantics.py --kernel
sudo python3 ipa/test/test_synth.py --kernel

# equivalenza NUMERICA sui sette modelli sintetici, nel kernel:
# float / int8 Python / int8 eBPF a confronto, 300 ingressi per scenario.
# La riga che conta e' "int8 (Python) vs int8 (eBPF)": deve dare 100%.
sudo python3 ipa/test/verify_synth_kernel.py --all --n 300
```

17.3 Datapath reale: il pacchetto attraversa davvero

Costruisce una rete di coppie veth, attacca la pipeline in modalità XDP native e verifica che il pacchetto esca dall'interfaccia giusta.

```
# tutte e tre le pipeline sullo stesso fabric
sudo python3 ipa/test/test_fabric.py

# una sola
sudo python3 ipa/test/test_fabric.py --method template

# confronto fra cammino native e cammino generic
sudo python3 ipa/test/test_fabric.py --xdp-mode generic

# se un run si interrompe e lascia interfacce in giro
sudo python3 ipa/test/netns_fabric.py --cleanup
```

17.4 Analisi parametrica: come cresce il costo

Sette assi storici più i quattro della campagna sulle architetture. Una cella per punto, ognuna in un processo separato perché un programma caricato non contamini il successivo — e perché un modello troppo largo sfonda lo stack eBPF con un abort che ucciderebbe l'intero sweep.

```
# i sette assi storici
sudo python3 ipa/test/bench_scaling.py

# uno alla volta: nodes, depth, width, descriptor,
# isoparam, degree, sparsity
sudo python3 ipa/test/bench_scaling.py --axis nodes
sudo python3 ipa/test/bench_scaling.py --axis depth --trials 15

# LA CAMPAGNA sulle architetture: iv_dense, iv_onehot,
```

```
# width_camp, depth_camp -- tutti in un CSV solo
sudo python3 ipa/test/bench_scaling.py --axis campaign --out results/

# con anche le letture di mappa per pacchetto (raddoppia
# le compilazioni: e' una seconda build strumentata)
sudo python3 ipa/test/bench_scaling.py --axis campaign --lookups --out results/

# salva i CSV (uno per asse storico)
sudo python3 ipa/test/bench_scaling.py --out results/

# i grafici dai CSV: non serve root, e non rimisura niente
python3 ipa/test/bench_scaling.py --plot results/
python3 ipa/test/bench_scaling.py --plot results/ --all-plots

# le pipeline danno tutte la stessa risposta? (80 confronti)
sudo python3 ipa/test/bench_scaling.py --verify

# larga contro profonda, a parita' di parametri
sudo python3 ipa/test/bench_depth_vs_width.py
sudo python3 ipa/test/bench_depth_vs_width.py --descriptor no_onehot
```

17.5 Il test reale: il costo del nodo

Il banco con generatore e nodo sotto prova (capitolo 13). **La forma da citare** è quella con il generatore a frame grezzi e l'uscita su un altro processore: misura il costo del nodo dalla ricezione all'inoltro, per tutte le pipeline più rxonly, nella stessa sessione.

```
sudo modprobe pktgen

# IL COSTO DEL NODO (la cifra da citare)
sudo python3 ipa/test/bench_throughput.py --mode compare \
  --generator xdp --egress-cpu 0 --rounds 5 \
  --out results/nodo 2>&1 | tee results/nodo.log

# lo stesso, per ogni classe che il modello sa decidere
sudo python3 ipa/test/bench_throughput.py --mode compare \
  --generator xdp --egress-cpu 0 --per-class --rounds 3 \
  --out results/nodo_classi

# lo stesso, su tre taglie di pacchetto
sudo python3 ipa/test/bench_throughput.py --mode compare \
  --generator xdp --egress-cpu 0 --frames 64,512,1514 --rounds 5

# con pktgen: anche la fase a carico fisso (respinti sotto capacita')
sudo python3 ipa/test/bench_throughput.py --mode compare --rounds 5

# le tre marcature di tempo T1/T2/T3 su una scala di rate
sudo python3 ipa/test/bench_throughput.py --mode rates \
  --frames 64 --rounds 3 --threads 2 --rates 0.5,1,1.5,2

# il generatore a frame grezzi contro pktgen, stesso ricevitore
sudo python3 ipa/test/xdp_gen.py --rounds 3

# la finestra di misura, verificata senza kernel (nessun root)
python3 ipa/test/test_steady_window.py

# se un run si interrompe e lascia il generatore configurato
sudo python3 ipa/test/bench_throughput.py --cleanup
```

Come si legge l'uscita

Costo per pacchetto sulla CPU del DUT — la tabella che conta: RX, elaborati, ns per pacchetto, differenza con la baseline, percentuale del tetto di sola ricezione. Il costo si legge dagli *elaborati*, non da RX.

Controllo di validità — la baseline deve essere la più veloce entro il 10% (nelle classi che inoltrano), e rxonly non deve essere più lento della baseline. Se una delle due cose non vale, il run misura la macchina e va rifatto.

Dispersione — sopra il 25% fra i giri la riga viene marcata: quel numero non è un risultato. Se succede, chiudere il resto sull'host e rilanciare.

17.6 Costi specifici

```
# quanto costa un salto fra programmi
sudo python3 ipa/test/bench_tailcall_overhead.py
sudo python3 ipa/test/bench_tailcall_overhead.py --repeat 5000 --trials 15

# quanto costa aggiungere un modello a caldo
sudo python3 ipa/test/bench_model_add.py
sudo python3 ipa/test/bench_model_add.py --n-models 3
```

17.7 Oggetto precompilato (AOT)

Genera il sorgente C dai pesi, lo compila e lo carica con un loader in C, senza Python né compilatore sul nodo.

```
make -C ipa/poc_aot          # genera, compila, linka
make -C ipa/poc_aot run      # compila e carica

# a mano, scegliendo il nodo e l'interfaccia
cd ipa/poc_aot
sudo ./loader_aot nn_aot_arch.o --node-id 7 --attach <ifindex>
sudo ./loader_aot nn_aot_arch.o --xdp-mode generic

make -C ipa/poc_aot clean
```

17.8 Diagnostica, quando qualcosa non torna

```
# che cosa decide il modello, al variare di ogni ingresso
python3 ipa/test/diag_model_decisions.py

# da quale commit P3 ha smesso di caricarsi; i tetti del verificatore
sudo python3 ipa/test/diag_p3_bisect.py --rev <commit> --rev HEAD
```

17.9 L'ordine consigliato

Partendo da zero su una macchina nuova:

1. `python3 ipa/test/test_suite.py` — il modello è sano?
2. `sudo python3 ipa/test/test_suite.py -only kernel` — si carica, e quanto costa?

3. `sudo python3 ipa/test/test_fabric.py` — il pacchetto attraversa davvero?
4. `sudo python3 ipa/test/bench_scaling.py -verify` — le tre pipeline concordano?
5. `sudo python3 ipa/test/bench_throughput.py -mode compare
-generator xdp -egress-cpu 0` — quanto costa un pacchetto al nodo?

I primi quattro rispondono *sì* o *no*. Solo il quinto produce un numero che va interpretato, ed è anche l'unico che può fallire per colpa della macchina invece che del codice.